REPÚBLICA BOLIVARIANA DE VENEZUELA
MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN SUPERIOR
UNIVERSIDAD NACIONAL EXPERIMENTAL
DE LAS TELECOMUNICACIONES E INFORMÁTICA

Pawcare: App móvil para atención veterinaria a domicilio

Miguel Figuera C.I: 23.558.789
Iromy León C.I: V-30.243.131
Alejandra Herde C.I: V-23.711.974

Caracas, Mayo de 2026

Nombre del proyecto

Pawcare: App móvil para atención veterinaria a domicilio.

Pawcare es una plataforma de gestión de relaciones con los clientes (CRM) para el consultorio veterinario Pawcare, cuyo componente central es una aplicación móvil que habilita la atención veterinaria a domicilio. El sistema digitaliza y optimiza la gestión de citas (en local o a domicilio), historias clínicas, control de vacunación, comunicación con los clientes y registro de pagos, sustituyendo los procesos manuales actuales por un canal digital accesible desde el teléfono del dueño y del personal de la clínica.

Alcance de la investigación

La investigación abarca el diseño de un sistema compuesto por un backend (API) y una aplicación móvil que sirve como canal de atención veterinaria a domicilio. La aplicación móvil permite a los dueños descubrir servicios, agendar y gestionar visitas (presenciales o a domicilio), consultar el historial clínico de sus mascotas, registrar pagos y dar seguimiento a vacunas y tratamientos; mientras que el personal autorizado gestiona pacientes, agenda, consultas y reportes desde el mismo dispositivo.

El dominio funcional considerado dentro del alcance incluye:

- Gestión de datos de clientes (dueños) y sus mascotas.
- Gestión integral de la agenda de citas, en local o a domicilio (creación, modificación, cancelación, recordatorios y confirmaciones).
- Gestión de esquemas de vacunación: recordatorios, costos y agenda.

- Gestión de la historia clínica electrónica (consultas, diagnósticos, tratamientos, vacunaciones, desparasitaciones y archivos adjuntos como exámenes).
- Prescripción de tratamientos y generación de récipes.
- Solicitud de exámenes de laboratorio.
- Registro básico de ingresos y egresos por servicios y productos vinculados a la consulta.
- Gestión de recordatorios (citas, vacunas, desparasitaciones).
- Gestión de servicios de atención (hospedaje, cirugía, hospitalización, grooming).
- Gestión de personal, roles y turnos.

Quedan fuera del alcance: la contabilidad financiera avanzada, el marketing y las campañas publicitarias complejas, la gestión avanzada de proveedores y compras, y la integración directa con equipos de laboratorio para la recepción automática de resultados.

En el plano técnico, la aplicación móvil se desarrolla para Android (Expo SDK 56 / React Native) y consume exclusivamente las 157 rutas del API documentadas para el cliente móvil, distribuidas en los dominios Público (19), Dueño/Owner (54) y Administrador/Admin (84).

Importancia de la investigación

El consultorio Pawcare, liderado por la Dra. Génesis Conesa, ofrece servicios de consulta, hospedaje y peluquería canina a precios accesibles, y constituye un pilar comunitario al participar con regularidad en campañas de vacunación y consultas veterinarias en servicios comunitarios. Atiende a una cantidad considerable de

pacientes de la ciudad de La Victoria, abarcando personas de distintos grupos socioeconómicos.

La importancia del proyecto radica en que la atención a domicilio, soportada por una aplicación móvil, acerca el servicio veterinario al hogar del paciente, fomenta la atención constante de las mascotas y favorece la detección temprana de enfermedades, el control de al menos dos consultas al año y los recordatorios de vacunación. Asimismo, resuelve problemas concretos del proceso manual actual:

- Gestión de citas manual, propensa a errores, duplicaciones y dificultades para optimizar la agenda.
- Historias clínicas en papel, con acceso lento, riesgo de extravío y dificultad para el seguimiento de enfermedades crónicas.
- Control de vacunación manual, con riesgo de olvidar fechas y refuerzos.
- Comunicación con clientes poco proactiva (recordatorios, seguimiento).
- Coordinación y registro de resultados de laboratorio de forma manual.
- Falta de diferenciación clara entre servicios (peluquería, hospedaje, consulta) y sus costos.
- Registro de pagos manual, sin visibilidad rápida del flujo de caja.

La participación activa de la Dra. Génesis y su asistente es fundamental para garantizar que la herramienta resultante sea verdaderamente útil y sostenible para la comunidad.

Informantes claves del diseño

Los informantes clave que alimentaron el diseño mediante el diagnóstico participativo son:

- **Dra. Génesis Conesa** — veterinaria principal y líder del consultorio Pawcare. Principal fuente sobre los procesos clínicos y administrativos, prioridades y expectativas.
- **Asistente del consultorio** — informante sobre la operación diaria, la gestión de citas y el registro de información.

Su intervención se canalizó a través de entrevistas individuales, observación directa del flujo de trabajo, análisis de los documentos existentes (formatos de historias clínicas y hojas de citas) y un taller participativo de validación de requerimientos.

Metodología de Ingeniería del software utilizada

El proyecto combina un enfoque de **diagnóstico participativo iterativo** con el marco ágil **Scrum** para el desarrollo del software.

El **diagnóstico participativo iterativo** estructura el levantamiento y la validación de requerimientos en fases que se recorren de forma incremental, con retroalimentación continua de los informantes clave:

1. **Preparación.** Reunión inicial, definición de roles y del alcance del CRM, elaboración de la guía de entrevista y de las plantillas de documentación.
2. **Recopilación de información.** Entrevistas individuales con la Dra. Génesis y su asistente, observación directa del flujo de trabajo y análisis de documentos existentes.
3. **Análisis y documentación de requerimientos.** Taller participativo, creación de casos de uso y priorización (alta, media, baja).
4. **Validación y ajuste.** Presentación de los requerimientos documentados, sesión de retroalimentación y ajuste según los comentarios.

5. **Desarrollo, implementación y evaluación.** Construcción del sistema, pruebas, capacitación, implementación y mejora continua.

Sobre esta base, el desarrollo se organiza con **Scrum**: el trabajo se divide en sprints cortos e iterativos, con un product backlog derivado de los requerimientos priorizados, reuniones de planificación y revisión por sprint, y entregas incrementales que se validan con el consultorio. Esta combinación permite incorporar la retroalimentación de los usuarios de forma temprana y frecuente, reduciendo el riesgo de construir funcionalidades que no respondan a las necesidades reales de la atención a domicilio.

Criterios básicos del diseño

La filosofía de diseño busca un sistema robusto, mantenible y fácil de instalar y desplegar, capaz de escalar desde una implementación inicial en un único servidor hasta una arquitectura distribuida conforme crezca la demanda. Los criterios básicos que guían el diseño son:

- **Robustez y mantenibilidad.** Diseño modular que facilita actualizaciones y pruebas automatizadas.
- **Facilidad de despliegue.** Empaquetado en contenedores (Docker) para una instalación reproducible.
- **Escalabilidad.** Capacidad de crecer en usuarios y volumen de datos sin re-arquitectura.
- **Mobile-first.** La interfaz se concibe primero para el teléfono (uso a una mano, orientación vertical, áreas táctiles ≥ 48 dp, tipografía ≥ 16 px y respeto de las áreas seguras).

- **Operación offline / offline-first.** La app permite consultar la última copia local y encolar acciones de escritura cuando no hay conexión, con sincronización bajo consentimiento del usuario.
- **Seguridad por roles.** Control de acceso diferenciado para los dominios público, dueño y administrador, con almacenamiento seguro de tokens y comunicación cifrada.
- **Consistencia visual.** Uso de un sistema de tokens de diseño (colores, espaciado, tipografía, sombras) con modo claro y oscuro.

Estilos arquitectónicos a utilizar

El sistema adopta una **arquitectura de tres capas contenerizada con Docker**, bajo un estilo **cliente-servidor**:

- **Capa de backend (servidor).** Implementada en Ruby on Rails en modo API (`api_only`), expone una **API RESTful** que entrega y recibe JSON. Internamente sigue el patrón **MVC** con la lógica de negocio encapsulada en clases de acción que devuelven un resultado explícito (**Action Pattern / Result**: `Result.success(...)` o `Result.failure(error:)`), manteniendo los controladores delgados. La autenticación es **JWT stateless**. Emplea ActiveRecord como ORM y ActiveStorage para archivos multimedia, además de borrado lógico (soft delete) y relaciones polimórficas.
- **Capa de cliente móvil.** Aplicación React Native (Expo SDK 56, TypeScript) organizada **por capas**: navegación → pantallas (orquestación) → componentes de UI → servicios (cliente HTTP) → hooks (lógica reutilizable) → tipos y tema. Es un cliente que consume la API por HTTPS y mantiene una cola local para la operación offline.

- **Capa de persistencia.** Base de datos relacional **MySQL** normalizada, con respaldos automáticos y volúmenes persistentes para multimedia.

La estrategia de escalamiento contempla una primera fase de escalamiento vertical (incremento de recursos del servidor e introducción de caché) y una segunda fase de escalamiento horizontal (CDN, balanceador de carga, flotas de servidores de frontend y backend, y base de datos en modo primario-réplica).

Trazabilidad del diseño

La trazabilidad asegura que cada necesidad quede cubierta por un artefacto de diseño y prueba, evitando requisitos huérfanos. Se mantienen los siguientes instrumentos:

- **Cadena requisito → ruta → pantalla → componente.** Cada requisito funcional se vincula con las rutas del API que lo implementan, las pantallas que lo presentan y los componentes de UI que lo construyen (criterio de mantenibilidad RNF-MAINT-001).
- **Apéndice de cobertura endpoint → requisito.** Las 157 rutas móviles se mapean a su requisito funcional; la cobertura es de **156 de 157** endpoints, con un único endpoint sin requisito (`GET /auth/debug`, de diagnóstico).
- **Matriz Requisito ↔ Caso de Uso ↔ Prueba.** Garantiza que todos los RF/RNF estén cubiertos por casos de uso y pruebas de aceptación.
- **Matrices de negocio.** Matriz Objetos vs. Procesos (CRUD) y matriz Actores/Unidades vs. Procesos (RACI) para ubicar responsabilidades.

Los documentos de requisitos y matrices se mantienen bajo control de versiones, lo que permite auditoría completa y reversión.

Atributos del diseño

Los atributos de calidad que orientan el diseño se alinean con el modelo **ISO/IEC 25010**:

- **Usabilidad.** Facilidad de aprendizaje (menos de 2 horas de formación), eficiencia (tareas comunes en 3–5 interacciones), satisfacción del usuario y prevención y recuperación de errores.
- **Rendimiento.** Transiciones entre pantallas en menos de 1 segundo en conexión normal; tiempos de respuesta acotados para cargar historiales, guardar entradas y buscar pacientes.
- **Escalabilidad.** Soporte de crecimiento de usuarios y datos sin re-arquitectura.
- **Disponibilidad y fiabilidad.** Alta disponibilidad en horario operativo, integridad de datos con transacciones atómicas, objetivos de respaldo y recuperación (RPO/RTO) y tolerancia a fallos con preservación de los datos del usuario.
- **Seguridad.** Control de acceso por roles, cifrado en tránsito y en reposo, pistas de auditoría y validación de entradas (prevención de inyección SQL y XSS).
- **Mantenibilidad y compatibilidad.** Modularidad, testeabilidad, compatibilidad con las versiones de Android soportadas por Expo SDK 56 y exportación de datos.

Requisitos funcionales

Los requisitos funcionales se documentan en los anexos y suman **26 requisitos** organizados por dominio.

Dominio Público (sin autenticación)

- **RF-PUB-01** — Información de servicios clínicos (Media).

- **RF-PUB-02** — Tienda de productos y checkout (Media).
- **RF-PUB-03** — Adopción de mascotas (Media).
- **RF-PUB-04** — Patrocinios, acceso público (Baja).
- **RF-PUB-05** — Contacto y contenido legal (Alta).

Dominio Dueño (Owner)

- **RF-OWN-01** — Autenticación y gestión de sesión (Alta).
- **RF-OWN-02** — Registro de dueño y recuperación de contraseña (Alta).
- **RF-OWN-03** — Perfil del dueño (Alta).
- **RF-OWN-04** — Dashboard del dueño (Alta).
- **RF-OWN-05** — Gestión de mascotas (Alta).
- **RF-OWN-06** — Citas veterinarias (Alta).
- **RF-OWN-07** — Pagos del dueño (Alta).
- **RF-OWN-08** — Historial médico, consultas y resumen (Alta).
- **RF-OWN-09** — Perfil médico, vacunas y desparasitaciones (Media).
- **RF-OWN-10** — Reportes médicos y exámenes de laboratorio (Media).
- **RF-OWN-11** — Patrocinios del dueño (Baja).

Dominio Administrador (Admin)

- **RF-ADM-01** — Gestión de mascotas (Alta).
- **RF-ADM-02** — Gestión de adopciones (Media).
- **RF-ADM-03** — Consultas clínicas (Alta).
- **RF-ADM-04** — Recetas y exámenes de laboratorio (Alta).
- **RF-ADM-05** — Vacunas, desparasitaciones y esquemas (Alta).

- **RF-ADM-06** — Agenda y citas (Alta).
- **RF-ADM-07** — Pagos (Alta).
- **RF-ADM-08** — Perfil médico de mascotas (Media).
- **RF-ADM-09** — Mascotas de dueños (Media).
- **RF-ADM-10** — Reportes médicos (Media).

Requisitos no funcionales

Los 17 requisitos no funcionales, también documentados en `docs/mobile-requirements.md`, se agrupan por categoría:

- **RNF-ARQ-001** — Plataforma móvil Android (Expo SDK 56, React Native, TypeScript).
- **RNF-ARQ-002** — Integración con API mediante un cliente HTTP centralizado (157 rutas).
- **RNF-MOB-001** — Mobile First desde 320 px de ancho lógico.
- **RNF-MOB-002** — Auto-sincronización bajo consentimiento del usuario (WiFi o datos).
- **RNF-SEC-001** — Seguridad de sesión y almacenamiento seguro de tokens (HTTPS).
- **RNF-SEC-002** — Control de acceso por rol y dominio (público, dueño, admin).
- **RNF-UX-001** — Experiencia de usuario móvil (táctil 48 dp, fuente ≥ 16 px, estados de carga/vacío/error).
- **RNF-UX-002** — Design system y tema claro/oscuro.
- **RNF-REL-001** — Tolerancia a fallos con preservación de datos.
- **RNF-REL-002** — Copia de seguridad local y recuperación (RPO/RTO).

- **RNF-REL-003** — Operación offline controlada.
- **RNF-PERF-001** — Rendimiento en red móvil (transiciones < 1 s, paginación).
- **RNF-ARC-001** — Manejo de archivos y multimedia (cámara, galería, PDF).
- **RNF-PRI-001** — Privacidad y datos de salud animal (mínima exposición).
- **RNF-OBS-001** — Observabilidad y registro de errores.
- **RNF-CMP-001** — Compatibilidad de dispositivos Android.
- **RNF-MAINT-001** — Mantenibilidad y trazabilidad documental.

Estándares de calidad empleados

El proyecto no persigue una certificación formal, pero se alinea con estándares y buenas prácticas reconocidas:

- **ISO/IEC 25010** — modelo de calidad del producto de software, marco de los atributos de calidad descritos.
- **IEEE 830** — estilo de especificación de requisitos de software (organización de RF y RNF).
- **Material Design** — guías de interacción y áreas táctiles (48 dp) para la interfaz Android.
- **HTTPS / TLS 1.2–1.3** — cifrado del transporte, con Perfect Forward Secrecy y certificados Let's Encrypt.
- **Política de contraseñas** — longitud mínima, combinación de mayúsculas, minúsculas y números, y bloqueo tras intentos fallidos.
- **Normas APA (7.ª edición)** — formato de la documentación del proyecto.
- **Convenciones de Rails y ActiveRecord** — integridad de datos, validaciones a nivel de modelo y auditoría de cambios.

Marco legal

El marco legal que sustenta el proyecto se compone de las siguientes normativas venezolanas:

- **Constitución de la República Bolivariana de Venezuela (1999),** Gaceta Oficial Extraordinaria N.° 5.453 del 24 de marzo de 2000. Artículo 110 (interés público de la ciencia, la tecnología y la innovación), Artículo 108 (incorporación de nuevas tecnologías en los centros educativos) y Artículo 98 (libertad de creación y divulgación científica, que respalda la publicación como software de código abierto).
- **Ley Orgánica de Ciencia, Tecnología e Innovación (LOCTI),** Gaceta Oficial N.° 38.242 del 3 de agosto de 2005 (reformas de 2010 y 2014). Artículos 1 y 2: generación de ciencia y tecnología al servicio de la soberanía nacional y declaratoria de interés público de estas actividades.
- **Ley de Infogobierno,** Gaceta Oficial N.° 40.274 del 17 de octubre de 2013. Artículos 1 y 34: promoción de las tecnologías de información libres y del conocimiento abierto, principios que inspiran la filosofía de código abierto del proyecto.
- **Ley de Ejercicio de la Medicina Veterinaria,** Gaceta Oficial del 19 de septiembre de 1968. Artículos 1 y 2: el ejercicio de la profesión se rige por la ley y las normas de ética del Colegio de Médicos Veterinarios, lo que orienta los procesos de registro clínico del sistema.
- **Código Deontológico de la Medicina Veterinaria,** emitido por la Federación de Colegios de Médicos Veterinarios de Venezuela (FCMVV).

- **Plan de la Patria 2025–2031** (Ley Orgánica, Gaceta Oficial N.° 6.907 Extraordinario del 24 de mayo de 2025) y **Plan de la Patria 2019–2025** (Gaceta Oficial Extraordinaria N.° 6.442 del 3 de abril de 2019), con los que el proyecto se vincula en materia de desarrollo tecnológico al servicio de la sociedad.

En materia de protección de datos, el diseño adopta de forma voluntaria principios alineados con normativas internacionales (LGPD/GDPR), como la minimización de la exposición de datos, el cifrado en tránsito y en reposo, y las pistas de auditoría.

Participantes investigadores y Organizaciones

Equipo investigador (autores):

- Miguel Figuera — C.I: 23.558.789.
- Iromy León — C.I: V-30.243.131.
- Alejandra Herde — C.I: V-23.711.974.

Tutor: Yuly Delgado.

Organizaciones:

- **Universidad Nacional Experimental de las Telecomunicaciones e Informática (UNETI)** — institución académica que enmarca el Proyecto Socio Tecnológico.
- **Consultorio Veterinario Peludog** — institución beneficiaria, dirigida por la Dra. Génesis Conesa (La Mora I, La Victoria, estado Aragua).
- **Comunidad de software libre de Ruby on Rails** — ecosistema técnico de soporte.

Comentarios

El presente documento se centra en el diseño del **backend (API)** y de la **aplicación móvil Android** que habilita la atención veterinaria a domicilio; se excluye expresamente el cliente web. Los requisitos funcionales y no funcionales provienen de la especificación móvil (`docs/mobile-requirements.md`), que cubre 157 rutas del API con una trazabilidad verificada de 156/157 endpoints hacia algún requisito. El diseño prioriza la operación offline y la sincronización bajo consentimiento, por ser determinantes para un servicio que ocurre fuera de la clínica.

Observaciones

- La validación final de requerimientos con el consultorio (Fase 4 del diagnóstico participativo) debe mantenerse viva en cada sprint para incorporar ajustes.
- Existe una diferencia de versiones del backend entre fuentes (Rails 7 en modo API y Rails 8); debe unificarse al iniciar la implementación.
- El escalamiento horizontal (CDN, balanceador, réplica de base de datos) se plantea como trabajo futuro, una vez superada la capacidad de la fase de escalamiento vertical.
- La marca del proyecto es **Pawcare**; cualquier referencia heredada a nombres anteriores debe normalizarse.

Referencias

Constitución de la República Bolivariana de Venezuela. (2000). Gaceta Oficial Extraordinaria N.° 5.453 del 24 de marzo de 2000.

Federación de Colegios de Médicos Veterinarios de Venezuela. (s. f.). Código Deontológico de la Medicina Veterinaria.

Ley de Ejercicio de la Medicina Veterinaria. (1968). Gaceta Oficial del 19 de septiembre de 1968.

Ley de Infogobierno. (2013). Gaceta Oficial N.° 40.274 del 17 de octubre de 2013.

Ley Orgánica de Ciencia, Tecnología e Innovación (LOCTI). (2005). Gaceta Oficial N.° 38.242 del 3 de agosto de 2005, con reformas de 2010 y 2014.

Plan de la Patria 2019–2025. Tercer Plan Socialista de Desarrollo Económico y Social de la Nación. (2019). Gaceta Oficial Extraordinaria N.° 6.442 del 3 de abril de 2019.

Plan de la Patria de las 7 Grandes Transformaciones 2025–2031. (2025). Ley Orgánica publicada en Gaceta Oficial N.° 6.907 Extraordinario del 24 de mayo de 2025.

ADR 0001 — Arquitectura de datos móvil

Estrategia **offline-first híbrida** · Pawcare — App móvil para atención veterinaria a domicilio

Aceptado 2026-05-31 Expo SDK 56 · React Native · TypeScript SQLite (expo-sqlite + Drizzle) redux-persist API REST Rails

Contexto y decisión

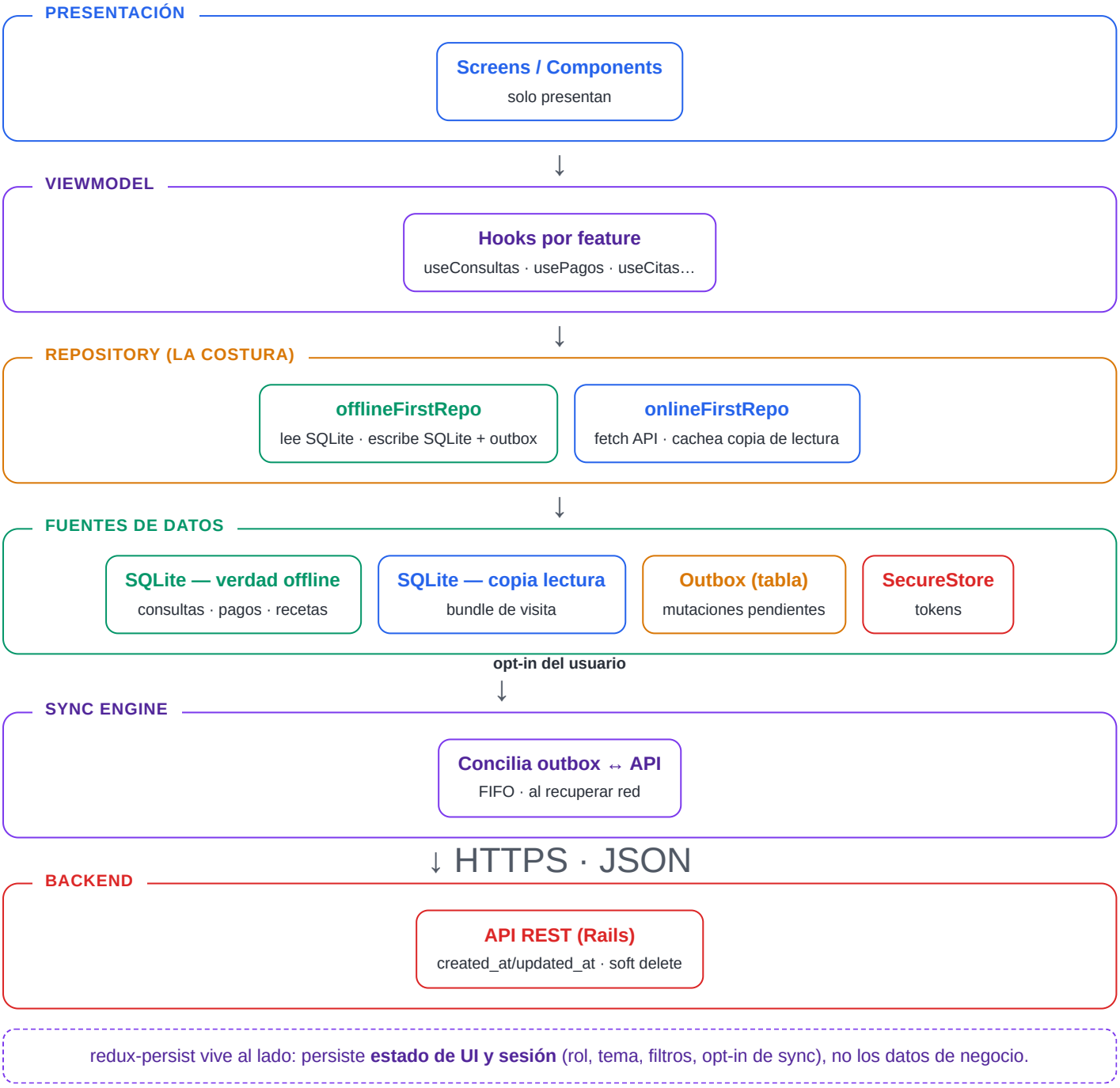
La app habilita **atención veterinaria a domicilio**: la conexión es mala justo donde ocurre el trabajo (la casa del paciente). No todo necesita operar sin red, pero los flujos de la visita —**registrar la consulta** y **registrar el pago recibido**— deben funcionar offline. Se adopta una arquitectura por capas con **Repository pattern** como costura entre la UI y las fuentes de datos, con una clasificación de datos en tres niveles.

> **Regla de oro:** no duplicar el mismo dato. **SQLite** = datos relacionales/consultables/grandes + verdad offline + copias de lectura + *outbox*. **redux-persist** = estado de UI y server-state pequeño (sesión, tema, filtros, opt-in). **SecureStore** = tokens y secretos.

Este documento explica, con diagramas: (2) las capas de datos, (3) la clasificación en 3 niveles, (4) el "bundle de visita" y la escritura offline, (5) el flujo del outbox/sincronización y (6) el reparto de almacenamiento.

1. Capas de datos

La UI nunca llama al API directamente: pide al repositorio, que decide local vs remoto.



redux-persist vive al lado: persiste **estado de UI y sesión** (rol, tema, filtros, opt-in de sync), no los datos de negocio.

2. Clasificación de datos en tres niveles

Qué se puede escribir offline, qué solo se cachea para lectura y qué exige conexión.

Nivel 1 · Offline-first

SQLite (fuente de verdad) + outbox · escritura offline

- **Consultas**
- **Recetas**
- **Vacunas**
- **Desparasitaciones**
- **Exámenes lab**
- **Adjuntos médicos**
- **Pagos recibidos ★**
- **Estado de cita (en visita)**
- **Borradores**

Nivel 2 · Online-first + copia

API; copia de solo-lectura en SQLite; escritura requiere red

- Citas (listado)
- Pagos (listado)
- Productos
- Dashboard
- Perfil
- Pacientes / Owners
- Servicios
- **Bundle de visita**

Nivel 3 · Online-only

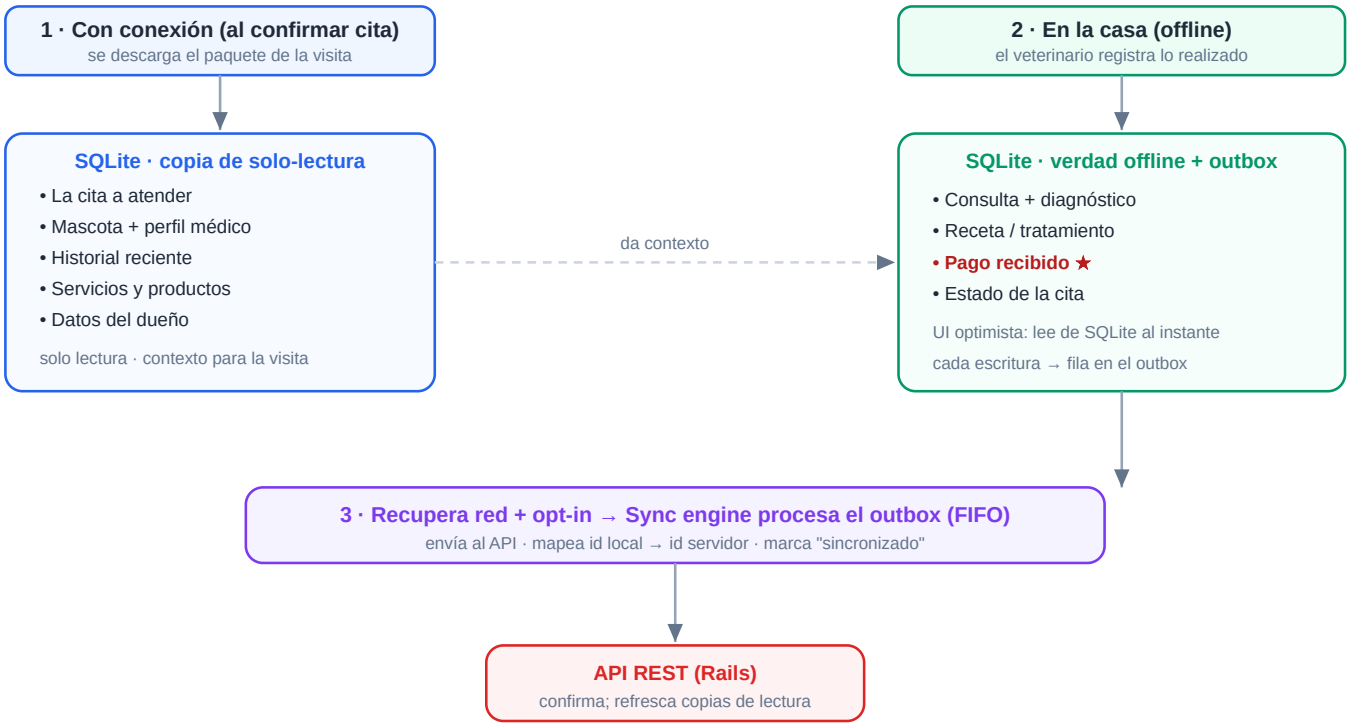
Sin valor offline; no se cachea

- Login / Registro
- Recuperar contraseña
- Generar reportes
- Exportar (CSV/PDF/JSON)
- Enviar por email

★ El **registro de pago recibido** fuera de la clínica es offline-first, igual que la consulta: el cobro ocurre en la casa.

3. "Bundle de visita" y escritura offline

Antes de la visita se replica a SQLite solo lo necesario (lectura). En la casa se crea consulta y pago (verdad local + outbox).



Hasta que el outbox sincroniza, la consulta y el pago se muestran como **"pendientes de sincronización"**; no se simula éxito en el servidor.

4. Flujo del outbox y la sincronización

Mismo camino para una consulta o un pago recibido. Rombos = decisión.



5. Reparto de almacenamiento

Cada almacén tiene un trabajo distinto. La disciplina evita duplicar datos.

SQLite (expo-sqlite + Drizzle)

- **Entidades offline-first**
- consultas · recetas · vacunas
- **pagos recibidos**
- **Copias de lectura (bundle)**
- **Outbox (tabla)**
- migraciones tipadas

redux-persist

- sesión / rol
- tema claro/oscuro
- filtros · paso del wizard
- opt-in de sync
- resumen del dashboard
- nada de listas grandes

SecureStore

- access token
- refresh token
- secretos

Si lo vas a consultar/filtrar o es grande → SQLite. Si es estado de pantalla pequeño → redux-persist. Si es un secreto → SecureStore.

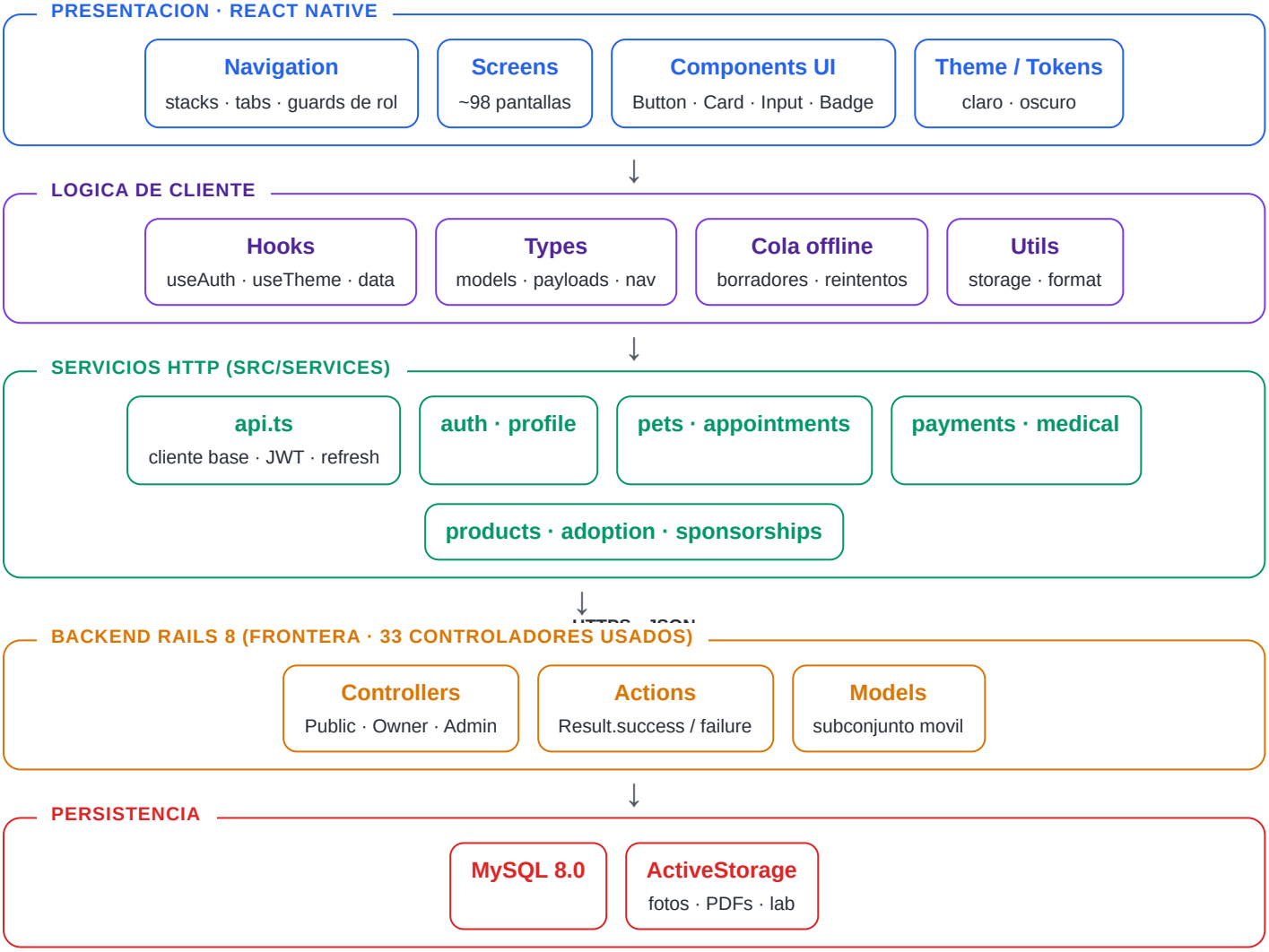
PawCare Mobile

Diagramas de arquitectura — aplicacion movil (Expo SDK 56 · React Native · TypeScript)

Alcance: este documento cubre **unicamente lo que consume la app movil** — las **157 rutas** de `mobile-routes.md` (Public 19 · Owner 54 · Admin 84), repartidas en **33 controladores** del backend Rails.
Excluido (solo web): metricas, turnos/shifts, modulo financiero, gestion de contenido y campanas de email, administracion de staff, movimientos de stock, categorias de servicio y configuracion. Esos endpoints existen en el backend pero la app movil no los usa.

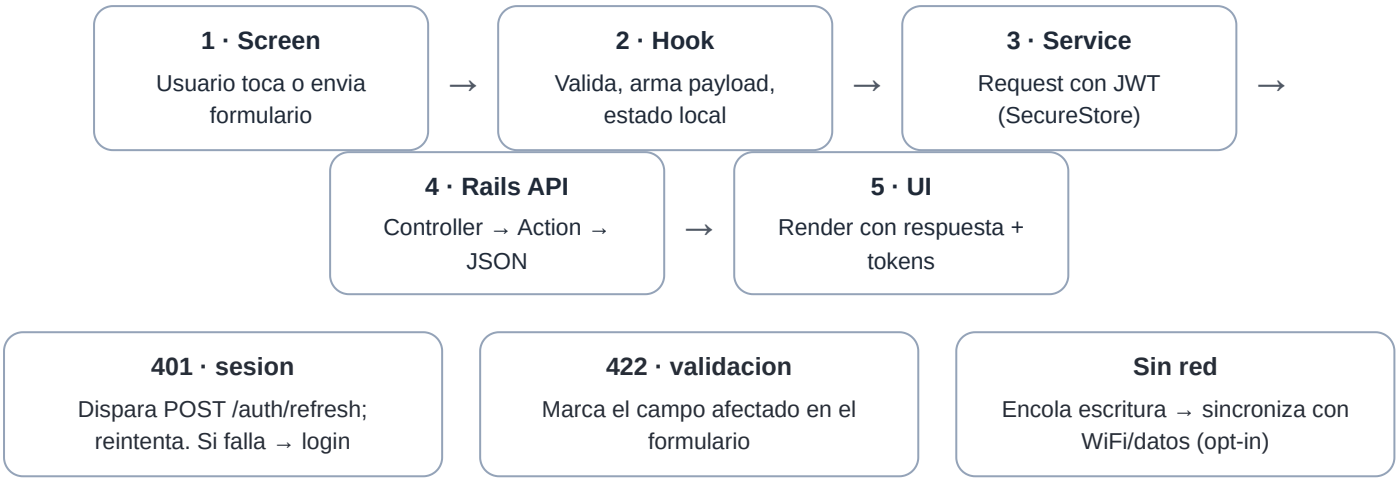
1. Diagrama de capas

Flujo vertical cliente → backend. La app movil es el cliente; el backend Rails se muestra solo como frontera de integracion.



2. Diagrama de flujo de una peticion

Ciclo de vida de una accion del usuario (ej. agendar cita, crear mascota).



Manejo de errores y operacion offline (RNF-REL / RNF-MOB-002).

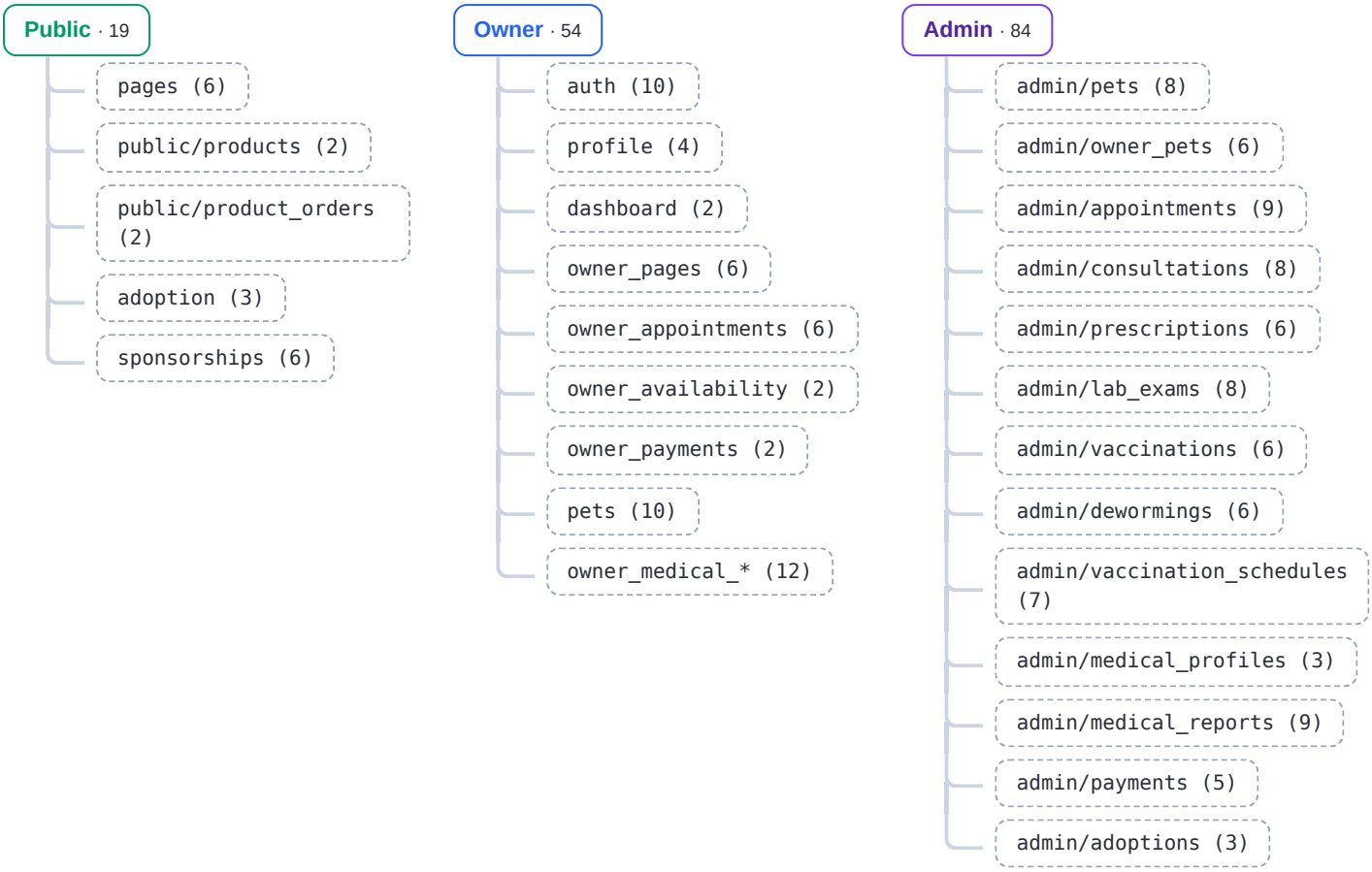
3. Diagrama de navegacion

RootNavigator decide el arbol segun el estado de sesion y el rol. Columna izquierda: acceso publico / pre-login. Columna derecha: navegadores autenticados (owner y staff).



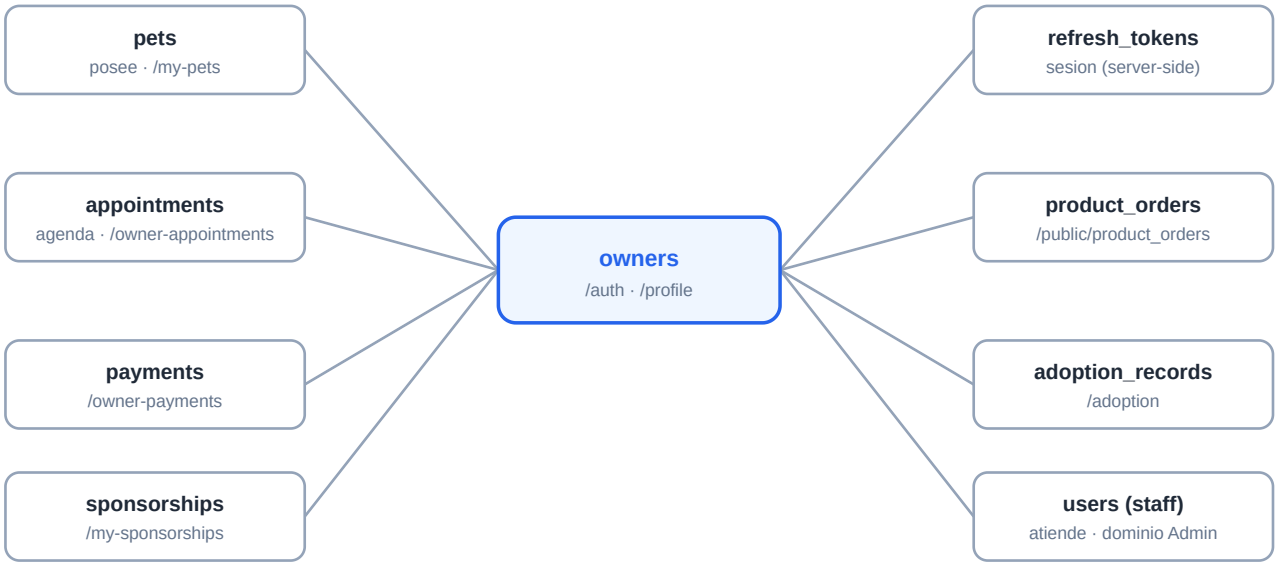
4. Diagrama de dominios y grupos de rutas

Las 157 rutas moviles agrupadas por dominio y controlador (numero de rutas entre parentesis).

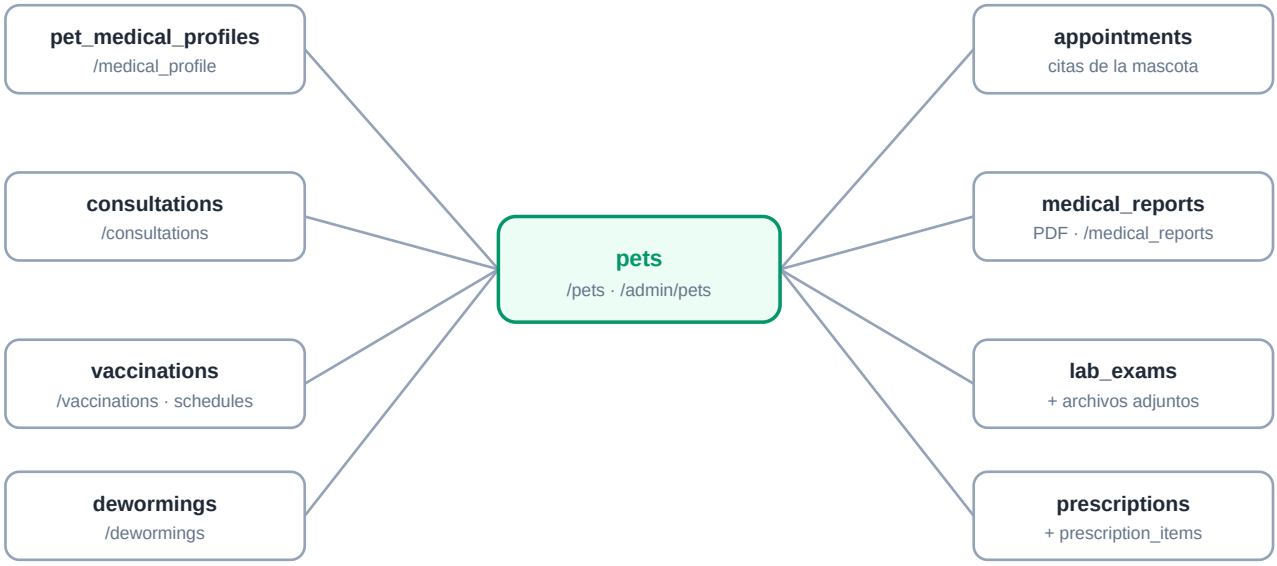


5. Diagrama de entidades (datos que toca la app movil)

Solo el subconjunto de tablas accesibles desde rutas moviles. Diagramas hub centrados en las dos entidades principales: Owner y Pet.



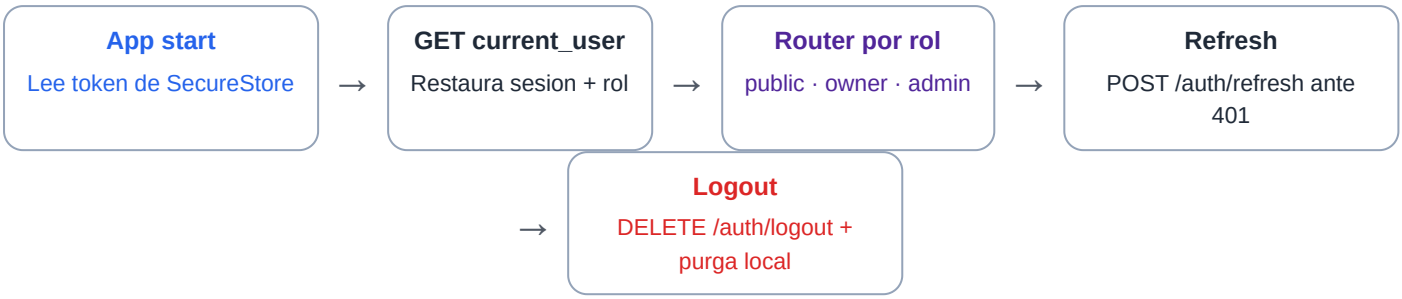
Hub 1 — **owners** y sus relaciones directas.



Hub 2 — **pets**: entidad clinica central. consultations agrupa recetas, examenes y tratamientos.

6. Diagrama de secuencia — sesion y seguridad

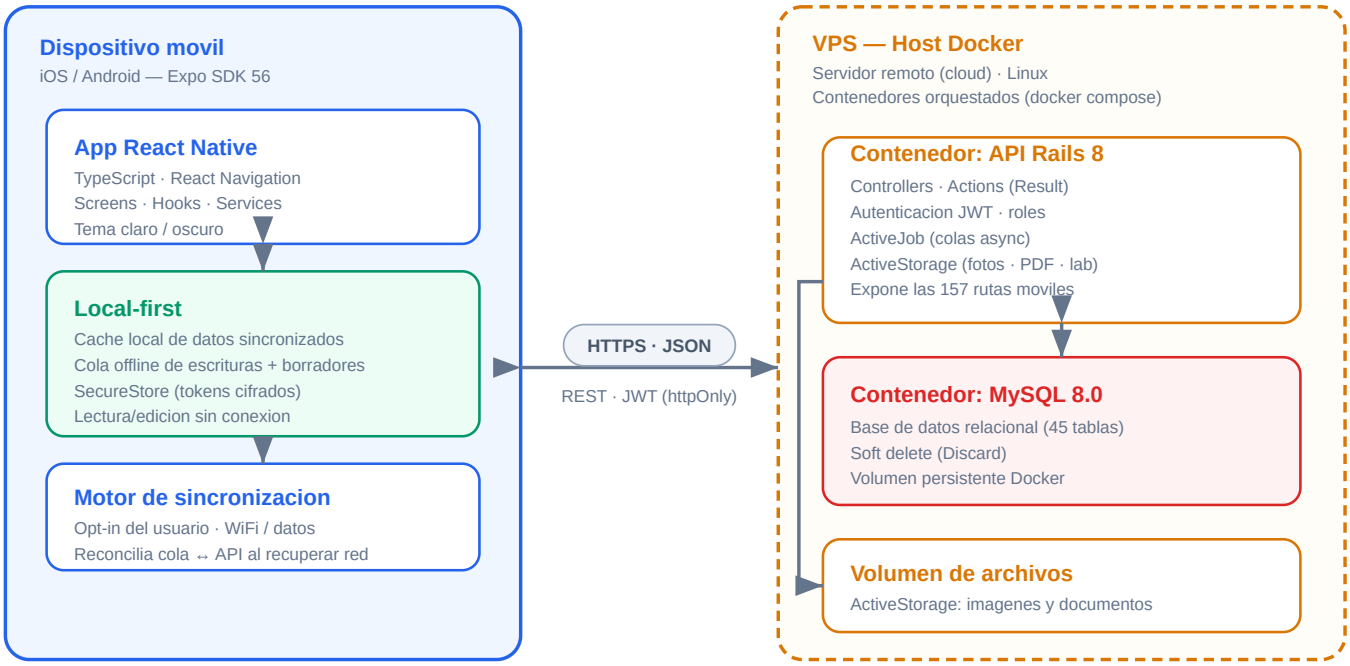
Manejo de tokens (RNF-SEC-001) y control de acceso por rol (RNF-SEC-002).



Un dueno nunca accede a pantallas ni endpoints /admin/*; intentos no autorizados → 403 / redireccion a login.

7. Diagrama de elementos del sistema (despliegue)

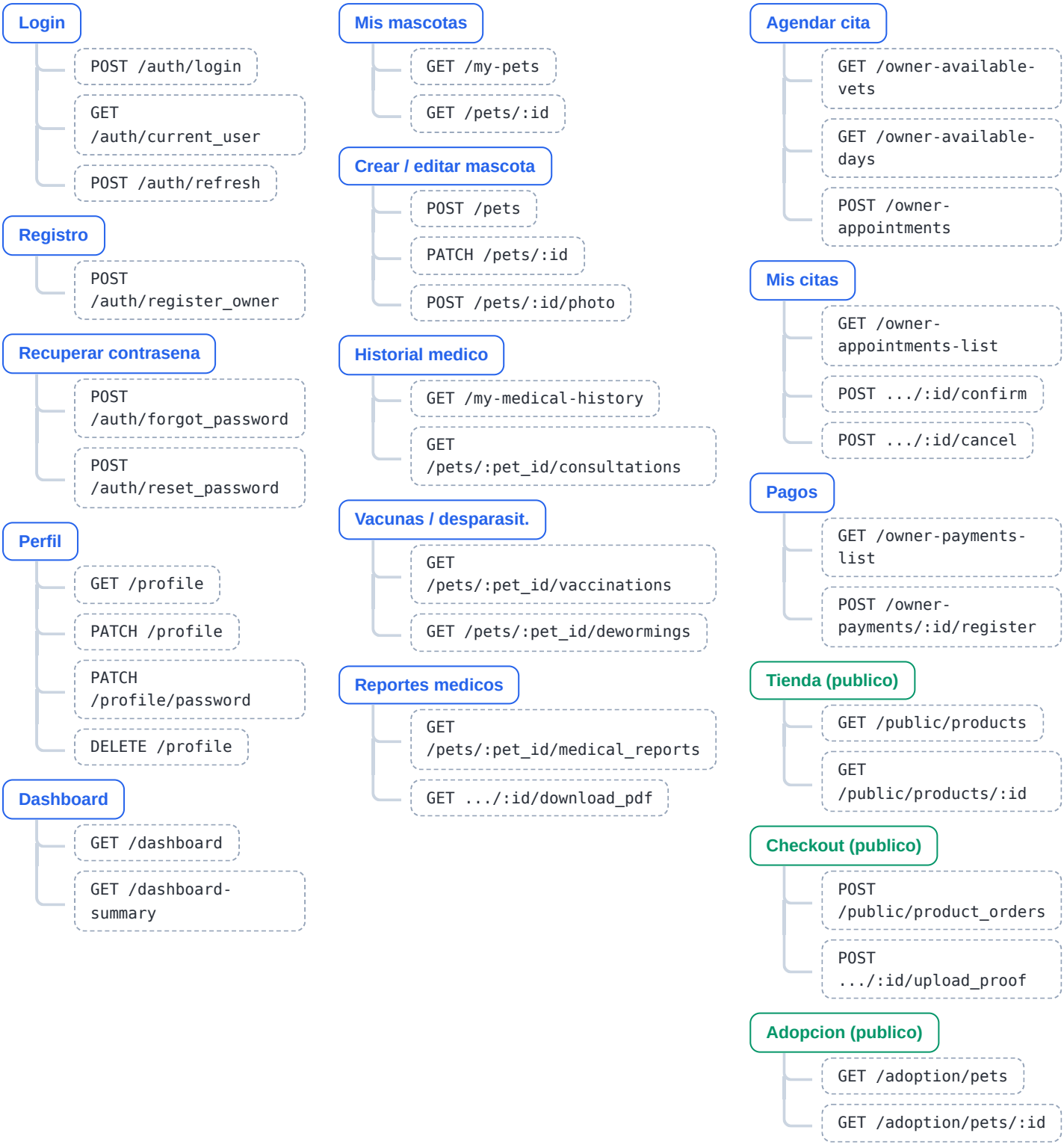
Nodos fisicos/logicos y su comunicacion. El dispositivo movil opera **local-first**; el backend Rails y MySQL corren en contenedores Docker.



El movil es **local-first**: funciona con su copia local y encola cambios; sincroniza con el backend por HTTPS solo con consentimiento del usuario (RNF-MOB-002 / RNF-REL-003). El backend Rails y MySQL se despliegan como contenedores Docker independientes sobre un **VPS** (servidor virtual remoto).

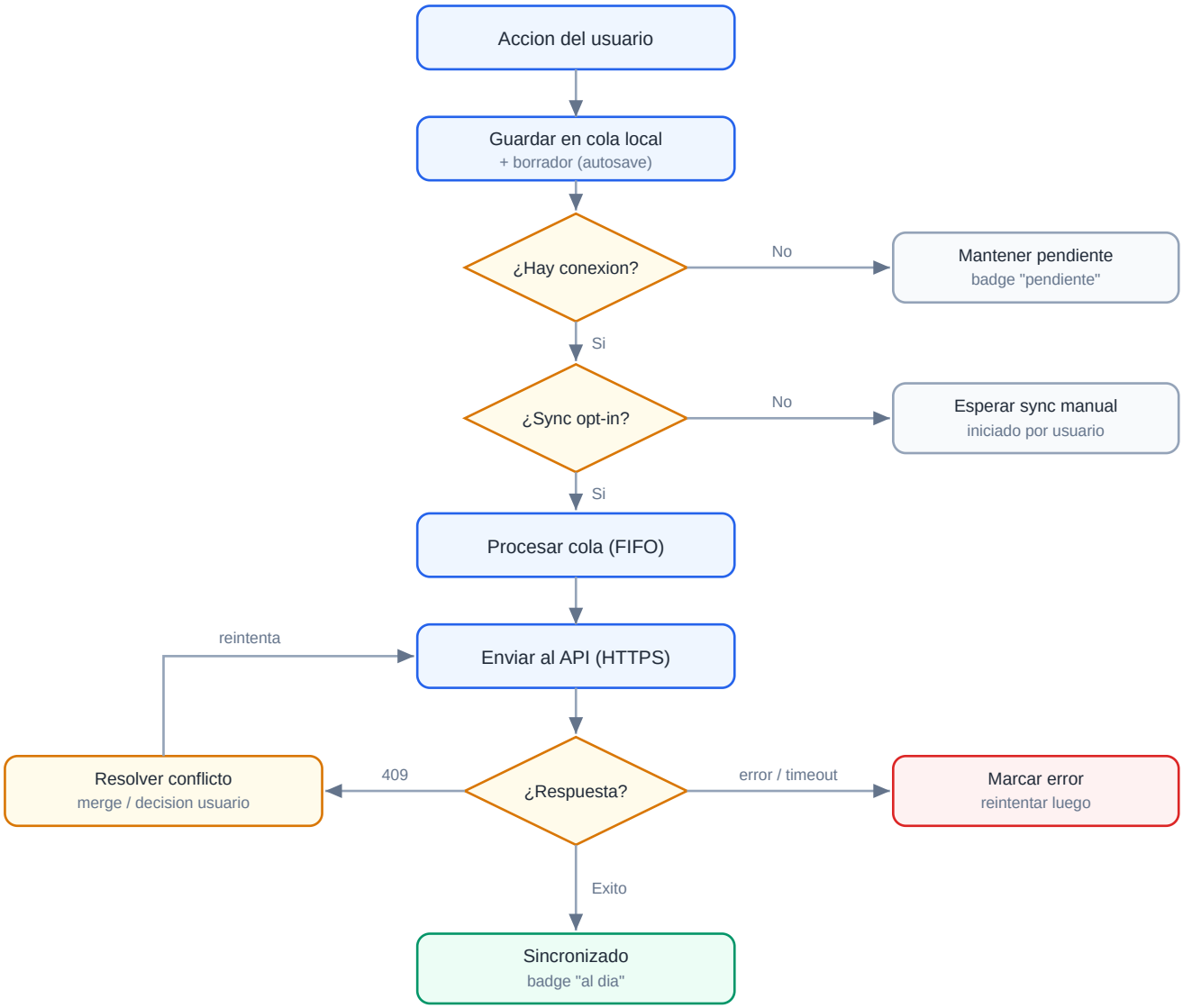
8. Mapa pantalla → endpoints

Cada pantalla principal con las rutas API que dispara. **Verde** = publico (sin auth) · **azul** = owner autenticado.



9. Diagrama de sincronizacion (local-first)

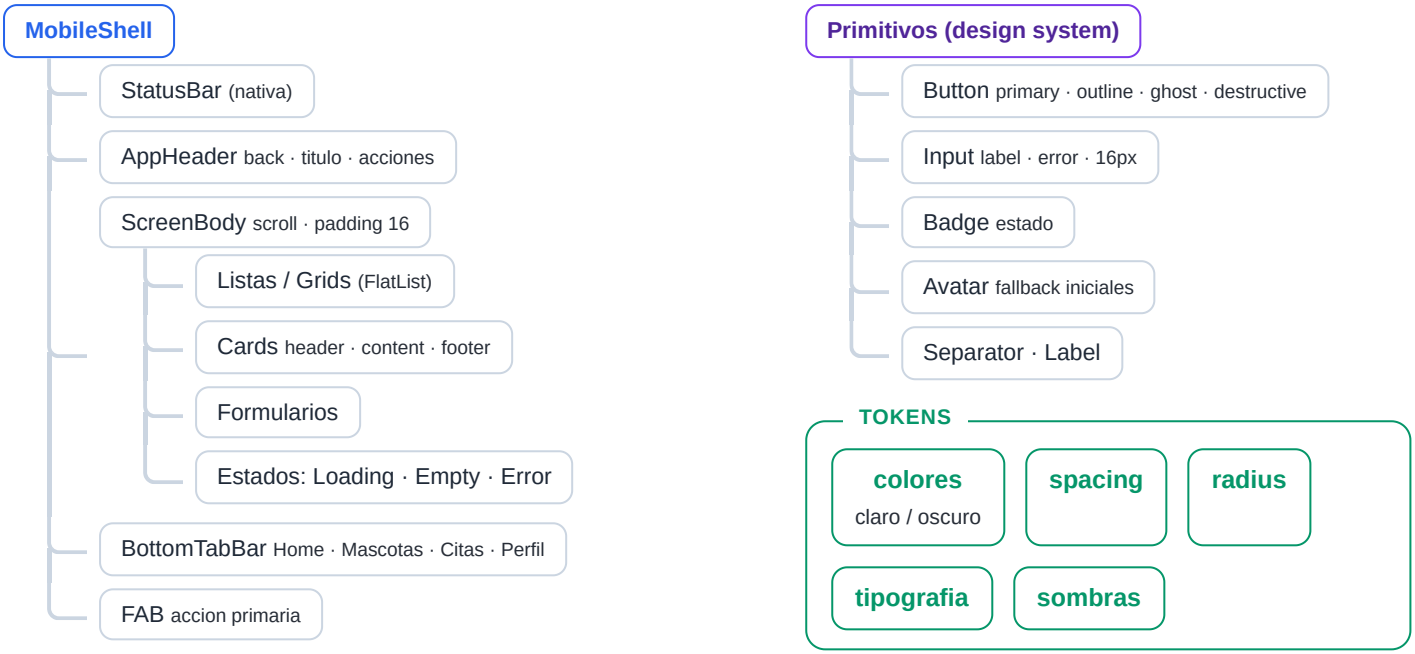
Flujo de la cola offline y el motor de sincronizacion (RNF-MOB-002, RNF-REL-002/003). Rombos = decision.



Sin conexion o sin opt-in, la app nunca simula exito: las escrituras quedan visibles como **pendientes** hasta confirmacion del servidor (RNF-REL-003).

10. Diagrama de componentes UI

Jerarquia de composicion (de `mobile-ui-components.md`). Los primitivos consumen los tokens del design system.



Los primitivos se construyen con `StyleSheet.create()` + `src/theme/tokens.ts` y alimentan todas las pantallas.

PawCare Mobile

Requisitos funcionales y no funcionales — aplicacion movil Android (Expo SDK 56 · React Native · TypeScript)

157 rutas API | 26 requisitos funcionales | 17 no funcionales | Public 19 · Owner 54 · Admin 84

Fuente: mobile-requirements.md

Convenciones. RF- (funcionales): Descripcion · Prioridad · Justificacion · Criterios de aceptacion. **RNF-** (no funcionales): `RNF-{Categoria}-{NNN}` + enunciado directo. Prioridades: Alta (MVP / legal) · Media (operacion completa) · Baja (mejora o alcance secundario).

Parte 1 — Requisitos funcionales

PUBLIC — SIN AUTENTICACION (RF-PUB)

RF-PUB-01 Informacion de servicios clinicos

Media

La app debe mostrar el catalogo de servicios veterinarios ofrecidos por la clinica, consumiendo la informacion publica del backend, sin requerir autenticacion.

Justificacion. Permite al usuario descubrir la oferta antes de registrarse o agendar una cita. Rutas: `GET /services`, `GET /pages/services`.

CRITERIOS DE ACEPTACION

- El usuario puede abrir la pantalla de servicios sin iniciar sesion.
- Se listan los servicios de `GET /services` con nombre, descripcion y precio cuando el API los provea.
- Ante error de red o respuesta vacia se muestra estado vacio o mensaje comprensible.
- Existe acceso claro a registro, login o agendamiento de cita desde servicios.

RF-PUB-02 Tienda de productos y checkout

Media

La app debe permitir consultar productos, ver detalle, crear un pedido y subir comprobante de pago, como flujo publico de compra.

Justificacion. Habilita venta de insumos desde el movil. Rutas: `GET /pages/products`, `GET /public/products`, `GET /public/products/:id`, `GET /pages/checkout`, `POST /public/product_orders`, `POST /public/product_orders/:id/upload_proof`.

CRITERIOS DE ACEPTACION

- Listar productos via `GET /public/products` y ver detalle via `GET /public/products/:id`.
- El checkout muestra resumen del pedido antes de confirmar.
- Al confirmar, envia `POST /public/product_orders` con los datos requeridos.
- Tras crear el pedido, adjuntar comprobante con `POST /public/product_orders/:id/upload_proof`.
- Mensajes de exito o error segun la respuesta; stock/indisponibilidad reflejado en UI.

RF-PUB-03 Adopcion de mascotas

Media

La app debe exponer el programa de adopcion: landing informativo, listado de mascotas disponibles y ficha de detalle.

Justificacion. Canal publico para promover adopciones responsables. Rutas: `GET /adoption`, `GET /adoption/pets`, `GET /adoption/pets/:id`.

CRITERIOS DE ACEPTACION

- Landing (`GET /adoption`) con informacion general y enlace al listado.
- Listado (`GET /adoption/pets`) con datos basicos (nombre, especie, edad, estado).
- Detalle (`GET /adoption/pets/:id`) con ficha completa, requisitos y accion de contacto/solicitud.
- Solo se muestran mascotas retornadas por el backend; sin datos hardcodeados.

RF-PUB-04 Patrocinios (acceso publico)

Baja

La app debe permitir consultar campanas de patrocinio, ver detalle, gastos asociados y crear o actualizar un patrocinio desde el dominio publico.

Justificacion. Canal de donaciones y transparencia de gastos. Rutas: `GET /sponsorships`, `GET /sponsorships/:id`, `GET /sponsorships/:id/expenses/:expense_id`, `POST /sponsorships`, `PATCH/PUT /sponsorships/:id`.

CRITERIOS DE ACEPTACION

- Listar patrocinios activos via `GET /sponsorships` ; detalle con meta, progreso e historial.
- Abrir un gasto via `GET /sponsorships/:id/expenses/:expense_id` .
- Alta via `POST /sponsorships` con validacion de campos obligatorios.
- Edicion via `PATCH` / `PUT` a `/sponsorships/:id` reflejada en pantalla.

RF-PUB-05 Contacto y contenido legal

Alta

La app debe mostrar paginas de contacto, terminos de uso y politica de privacidad como contenido informativo publico.

Justificacion. Requisito legal y de confianza para usuarios y tiendas de aplicaciones. Rutas: `GET /pages/contact`, `GET /pages/terms`, `GET /pages/privacy`.

CRITERIOS DE ACEPTACION

- Las tres pantallas accesibles sin autenticacion; contenido desde rutas `pages#*` .
- Contacto muestra medios oficiales (telefono, email, direccion) cuando el API los provea.
- Terminos y privacidad legibles en movil (scroll, tipografia adecuada).
- Enlaces a terminos y privacidad disponibles desde registro y checkout.

OWNER — DUENO DE MASCOTA (RF-OWN)

RF-OWN-01 Autenticacion y gestion de sesion

Alta

La app debe permitir login, logout, consulta del usuario autenticado y renovacion silenciosa de sesion mediante tokens.

Justificacion. Base de todo el dominio Owner y Admin. Rutas: `GET /auth`, `POST /auth/login`, `DELETE /auth/logout`, `GET /auth/current_user`, `POST /auth/refresh`.

CRITERIOS DE ACEPTACION

- Pantalla de bienvenida (`GET /auth`) con acceso a login y registro.
- Login envia credenciales a `POST /auth/login` y persiste el token de forma segura.
- Tras login, navega al dashboard/pantalla principal del rol correspondiente.
- `GET /auth/current_user` al iniciar para restaurar sesion valida.
- `POST /auth/refresh` antes de expirar o ante 401 recuperable, sin interrumpir al usuario.
- Logout (`DELETE /auth/logout`) invalida sesion local y remota.
- Credenciales invalidas muestran error sin revelar si el email existe.

RF-OWN-02 Registro de dueno y recuperacion de contrasena

Alta

La app debe permitir registrar un dueno de mascota y completar el flujo de olvido y restablecimiento de contrasena.

Justificacion. Onboarding de nuevos usuarios y recuperacion de acceso. Rutas: `POST /auth/register_owner`, `POST /auth/forgot_password`, `GET /auth/reset_password/:token`, `POST /auth/reset_password`.

CRITERIOS DE ACEPTACION

- Registro via `POST /auth/register_owner` con validacion client-side de email, password y campos obligatorios.
- Tras registro, el usuario inicia sesion o queda autenticado segun respuesta del API.
- "Olvide mi contrasena" envia email via `POST /auth/forgot_password` y confirma en UI.
- El deep link con token abre pantalla de nueva contrasena (`GET /auth/reset_password/:token`).
- Restablecer via `POST /auth/reset_password`; maneja token invalido o expirado.
- Passwords cumplen reglas minimas del backend.

RF-OWN-03 Perfil del dueno

Alta

El dueno autenticado debe poder ver, editar su perfil, cambiar contrasena y eliminar su cuenta.

Justificacion. Autogestion de datos personales y cumplimiento de derechos del usuario. Rutas: `GET /profile`, `PATCH /profile`, `PATCH /profile/password`, `DELETE /profile`.

CRITERIOS DE ACEPTACION

- `GET /profile` muestra datos actuales; edicion via `PATCH /profile`.
- Cambio de contrasena via `PATCH /profile/password` pidiendo actual y nueva.
- Eliminar cuenta requiere confirmacion explicita antes de `DELETE /profile`.
- Tras eliminar, la sesion se cierra y no quedan tokens locales.

RF-OWN-04 Dashboard del dueno

Alta

La app debe presentar un resumen del estado del dueno: citas proximas, pagos, alertas medicas y accesos rapidos.

Justificacion. Punto de entrada principal post-login. Rutas: `GET /dashboard`, `GET /dashboard-summary`.

CRITERIOS DE ACEPTACION

- Consume `GET /dashboard` y/o `GET /dashboard-summary`.
- Muestra al menos: proxima cita, cantidad de mascotas y pagos pendientes cuando el API los incluya.
- Accesos rapidos a mascotas, citas, historial medico y pagos.
- Solo accesible con sesion de dueno valida; ante fallo, estado de error con reintento.

RF-OWN-05 Gestion de mascotas (dueno)

Alta

El dueno debe poder listar, crear, editar, eliminar sus mascotas y gestionar la foto de perfil de cada una.

Justificacion. Entidad central del ecosistema PawCare. Rutas: `GET /my-pets`, `GET /pets`, `GET /pets/:id`, `GET /pets/new`, `GET /pets/:id/edit`, `POST /pets`, `PATCH/PUT /pets/:id`, `DELETE /pets/:id`, `POST /pets/:id/photo`, `DELETE /pets/:id/photo`.

CRITERIOS DE ACEPTACION

- Listado via `GET /my-pets` o `GET /pets`; creacion via `POST /pets` con campos validados.
- Edicion via `PATCH` / `PUT`; eliminacion via `DELETE` con confirmacion.
- Foto via `POST` / `DELETE /pets/:id/photo` desde camara o galeria.
- El dueno solo ve/modifica sus mascotas (403/404 manejados); desde detalle accede a historial y citas.

RF-OWN-06 Citas veterinarias (dueno)

Alta

El dueno debe poder listar citas, ver detalle, agendar, editar, confirmar y cancelar citas usando disponibilidad de veterinarios y dias.

Justificacion. Funcionalidad core de la clinica movil. Rutas: `GET /my-appointments`, `GET /owner-appointments-list`, `GET /owner-appointments/:id`, `POST /owner-appointments`, `PATCH /owner-appointments/:id`, `POST .../confirm`, `POST .../cancel`, `GET /owner-available-vets`, `GET /owner-available-days`.

CRITERIOS DE ACEPTACION

- Listado via `GET /owner-appointments-list` o hub `GET /my-appointments`; detalle con estado, mascota, vet y fecha/hora.
- Agendar via `POST /owner-appointments` tras seleccionar mascota, vet y dia/hora (endpoints de disponibilidad).
- Editar via `PATCH`; confirmar via `.../confirm`; cancelar via `.../cancel` con confirmacion.
- Citas pasadas y proximas distinguibles; no se permiten slots no disponibles.

RF-OWN-07 Pagos del dueno

Alta

El dueno debe consultar sus pagos pendientes o realizados y registrar comprobante de pago.

Justificacion. Cierra el ciclo de servicios facturables. Rutas: `GET /my-payments`, `GET /owner-payments-list`, `POST /owner-payments/:id/register`.

CRITERIOS DE ACEPTACION

- Listado via `GET /owner-payments-list` o hub `GET /my-payments`.
- Cada pago muestra monto, concepto, estado y fecha de vencimiento si aplica.
- Registrar pago via `POST /owner-payments/:id/register` con datos y comprobante.
- Tras registro, el estado se actualiza en listado y detalle; pagos ajenos no accesibles.

RF-OWN-08 Historial medico, consultas y resumen

Alta

El dueno debe consultar el historial medico global y por mascota, ver consultas, exportar recetas y marcar tratamientos completados.

Justificacion. Transparencia clinica y continuidad del cuidado. Rutas: `GET /my-medical-history`, `GET /my-pets/:pet_id/medical-history`, `GET /owner-medical-summary`, `GET /pets/:pet_id/consultations`, `GET /owner/consultations/:id/export_recipe`, `POST /owner/consultations/:id/complete_treatment`.

CRITERIOS DE ACEPTACION

- Hub via `GET /my-medical-history` y resumen via `GET /owner-medical-summary`.
- Historial por mascota y listado de consultas via las rutas correspondientes.
- Exportar receta via `export_recipe` y abrir/compartir el archivo.
- Completar tratamiento via `complete_treatment` con confirmacion; solo registros del dueno.

RF-OWN-09 Perfil medico, vacunas y desparasitaciones

Media

El dueno debe consultar el perfil medico base de su mascota, el historial de vacunas y desparasitaciones.

Justificacion. Informacion preventiva de salud animal. Rutas: `GET /pets/:pet_id/medical_profile`, `GET /pets/:pet_id/vaccinations`, `GET /pets/:pet_id/dewormings`.

CRITERIOS DE ACEPTACION

- Perfil medico via `medical_profile` (alergias, peso, notas segun API).
- Vacunas y desparasitaciones listadas con fechas y tipo.
- La UI indica proximas dosis o vencimientos cuando el backend los calcule.

RF-OWN-10 Reportes medicos y examenes de laboratorio (dueno)

Media

El dueno debe consultar reportes medicos en PDF, subir archivos de laboratorio y consultar resultados cuando el flujo lo permita.

Justificacion. Acceso documental y participacion en estudios clinicos. Rutas: `GET /pets/:pet_id/medical_reports`, `GET .../:id`, `GET .../:id/download_pdf`, `POST /owner/lab_exams/:id/files`, `PATCH /owner/lab_exams/:id/results`.

CRITERIOS DE ACEPTACION

- Listado y detalle de reportes; descarga PDF con apertura en visor o app externa.
- Subida de archivos de lab via `files`; actualizacion de resultados via `results` cuando corresponda al rol.
- Adjuntos muestran progreso de carga y errores de tamano o formato.

RF-OWN-11 Patrocinios del dueno

Baja

El dueno autenticado debe ver el listado de sus patrocinios activos desde su area personal.

Justificacion. Continuidad del flujo de donaciones en sesion autenticada. Ruta: `GET /my-sponsorships`.

CRITERIOS DE ACEPTACION

- Pantalla accesible desde dashboard o menu del dueno.
- Consume `GET /my-sponsorships` y lista solo patrocinios del usuario.
- Desde cada item se navega al detalle reutilizando RF-PUB-04.

ADMIN — PANEL ADMINISTRATIVO (RF-ADM)

RF-ADM-01 Gestion de mascotas (administrador)

Alta

El staff autorizado debe listar, crear, editar, eliminar mascotas clinicas y gestionar sus fotos desde el panel admin movil.

Justificacion. Operacion diaria de la clinica. Rutas: `GET /admin/pets-list`, `GET /admin/pets/:id`, `POST /admin/pets`, `PATCH/PUT /admin/pets/:id`, `DELETE /admin/pets/:id`, `POST/DELETE /admin/pets/:id/photo`.

CRITERIOS DE ACEPTACION

- Listado con busqueda via `GET /admin/pets-list`; CRUD completo segun permisos del rol.
- Gestion de foto via POST/DELETE en `/admin/pets/:id/photo`.
- Solo usuarios admin autenticados acceden; acciones destructivas con confirmacion.

RF-ADM-02 Gestion de adopciones (administrador)

Media

El staff debe listar solicitudes o registros de adopcion, ver detalle y crear nuevos registros de adopcion.

Justificacion. Gestion interna del programa de adopcion. Rutas: `GET /admin/adoptions-list`, `GET /admin/adoptions/:id`, `POST /admin/adoptions`.

CRITERIOS DE ACEPTACION

- Listado y detalle via las rutas correspondientes; alta via `POST` con validacion.
- Estados de adopcion mostrados segun respuesta del API.

RF-ADM-03 Consultas clinicas (administrador)

Alta

El staff debe gestionar consultas: listar, crear, editar, eliminar, exportar receta y completar tratamientos.

Justificacion. Nucleo del trabajo veterinario en la app. Rutas: `/admin/consultations*` (list, show, create, update, destroy), `.../export_recipe`, `.../complete_treatment`.

CRITERIOS DE ACEPTACION

- Listado filtrable; formulario envia POST o PATCH segun creacion o edicion.
- Detalle con notas clinicas, mascota, veterinario y estado.
- Exportar receta y completar tratamiento via sus endpoints; eliminar con confirmacion.

RF-ADM-04 Recetas y examenes de laboratorio (administrador)

Alta

Dentro de una consulta, el staff debe gestionar prescripciones y examenes de laboratorio, incluyendo archivos adjuntos.

Justificacion. Documentacion clinica completa en consulta. Rutas prescriptions y lab_exams bajo `/admin/consultations/:consultation_id/`.

CRITERIOS DE ACEPTACION

- CRUD de prescripciones y de examenes via los endpoints anidados.
- Subir archivos via `.../lab_exams/:id/files`; eliminar via `.../files/:file_id`.
- Cambios en receta/lab reflejados al volver al detalle de consulta.

RF-ADM-05 Vacunas, desparasitaciones y esquemas

Alta

El staff debe administrar registros de vacunacion, desparasitacion y esquemas programados, incluyendo marcar items completados.

Justificacion. Control preventivo de salud animal. Rutas: `/admin/vaccinations*`, `/admin/dewormings*`, `/admin/vaccination_schedules*`, `POST .../complete_item`.

CRITERIOS DE ACEPTACION

- CRUD de vacunas, desparasitaciones y esquemas via sus endpoints y listados `-list`.
- Completar item de esquema via `complete_item`.
- Listados muestran mascota asociada y fechas relevantes.

RF-ADM-06 Agenda y citas (administrador)

Alta

El staff debe gestionar la agenda: listar citas, crear y editar citas usando disponibilidad de veterinarios, dias y franjas horarias.

Justificacion. Coordinacion operativa de la clinica. Rutas: `/admin/appointments*` (list, show, available-days, available-vets, time-slots, create, update, destroy).

CRITERIOS DE ACEPTACION

- Listado/calendario via `GET /admin/appointments-list`.
- Creacion usa vets, dias y slots de disponibilidad antes de `POST`.
- Editar/cancelar via PATCH/PUT/DELETE; detalle con dueno, mascota, servicio y sala si aplica.
- Conflictos de horario retornados por el API se muestran al usuario.

RF-ADM-07 Pagos (administrador)

Alta

El staff debe listar pagos, ver detalle, registrar pagos recibidos y actualizar su estado.

Justificacion. Control financiero de servicios. Rutas: `/admin/payments-list`, `/admin/payments/:id`, `POST .../register`, `PATCH/PUT .../:id`.

CRITERIOS DE ACEPTACION

- Listado con filtros por estado si el API los soporta; detalle via `GET`.
- Registro de pago via `register`; actualizacion via PATCH/PUT.
- Comprobantes adjuntos visibles cuando el backend los incluya.

RF-ADM-08 Perfil medico de mascotas (administrador)

Media

El staff debe consultar y editar el perfil medico base de una mascota bajo administracion.

Justificacion. Datos clinicos de referencia para consultas. Rutas: `GET/PATCH/PUT /admin/pets/:pet_id/medical_profile`.

CRITERIOS DE ACEPTACION

- Visualizacion via GET; edicion via PATCH/PUT con validacion de campos clinicos.
- Cambios persisten y se reflejan en consultas posteriores.

RF-ADM-09 Mascotas de duenos (administrador)

Media

El staff debe poder crear y editar mascotas asociadas a un dueno especifico, incluyendo foto.

Justificacion. Alta asistida en recepcion o consulta. Rutas: `/admin/owners/:owner_id/pets*` (show, create, update, photo).

CRITERIOS DE ACEPTACION

- Creacion via `POST /admin/owners/:owner_id/pets`; edicion via PATCH/PUT anidada con owner_id.
- Gestion de foto via endpoints de photo; detalle via GET con datos del dueno y la mascota.

RF-ADM-10 Reportes medicos (administrador)

Media

El staff debe listar reportes medicos, generarlos, importarlos, exportarlos en multiples formatos y enviarlos por email.

Justificacion. Documentacion formal y comunicacion con duenos. Rutas: `/admin/medical_reports` (index, show, create, generate_pdf, import, export_csv/json/pdf, email).

CRITERIOS DE ACEPTACION

- Listado y detalle; generar PDF via `generate_pdf` ; importar via `import` .
- Exportar CSV, JSON y PDF via endpoints `export_*` ; enviar por email via `email` .
- Archivos generados se pueden abrir o compartir desde el dispositivo.

Parte 2 — Requisitos no funcionales

Formato `RNF-{Categoria}-{NNN}: Titulo` . Categorias: ARQ (arquitectura), MOB (movil), SEC (seguridad), UX (experiencia), REL (confiabilidad), PERF (rendimiento), ARC (archivos), PRI (privacidad), OBS (observabilidad), CMP (compatibilidad), MAINT (mantenibilidad).

RNF-ARQ-001 Plataforma movil Android

ARQ

La aplicacion debe desarrollarse con **Expo SDK 56**, **React Native** y **TypeScript**, como cliente nativo **solo para Android**. El codigo de pantallas y servicios debe estar tipado de forma estricta. Los flujos principales no deben depender de contenedores web embebidos. La version de Expo debe mantenerse alineada con la documentacion oficial v56 (`AGENTS.md` , `docs/arq.md`). No se incluyen targets, builds ni dependencias especificas de iOS.

RNF-ARQ-002 Integracion con API Rails

ARQ

La app debe consumir exclusivamente las **157 rutas** documentadas en `mobile-routes.md` , sin reimplementar logica de negocio en el cliente. Debe existir un cliente HTTP centralizado con base URL configurable por entorno (desarrollo, staging, produccion). Los payloads y respuestas deben estar tipados segun los contratos del API Rails existente.

RNF-MOB-001 Mobile First

MOB

La aplicacion debe disenarse y construirse con enfoque **mobile first**: layouts, navegacion, tipografia y patrones de interaccion pensados primero para pantallas de telefono (una mano, orientacion vertical, safe areas). Las adaptaciones a tablet, si existen en el futuro, deben ser extensiones del diseno movil base y no condicionar la arquitectura. Toda pantalla de `mobile-screens.md` debe priorizar legibilidad y acciones principales en viewports desde **320 px** de ancho logico.

RNF-MOB-002 Auto-sincronizacion bajo consentimiento del usuario

MOB

La aplicacion debe ser capaz de **auto-sincronizarse** con el backend cuando detecte conexion activa por **WiFi o datos moviles**, **si y solo si el usuario lo ha autorizado** previamente en ajustes (opt-in explicito, revocable en cualquier momento).

COMPORTAMIENTO ESPERADO

- Sin autorizacion, **ninguna** sincronizacion automatica en segundo plano; solo sincronizacion manual.
- Con autorizacion, al detectar conectividad procesa la cola local (formularios, uploads, acciones diferidas) y refresca datos stale segun prioridad.
- El usuario elige, como minimo, sincronizar solo con **WiFi** o tambien con **datos moviles**.
- Indicador discreto de estado (sincronizando / al dia / pendiente / error) sin interrumpir salvo conflicto que requiera decision.

RNF-SEC-001 Seguridad de sesion y almacenamiento de tokens

SEC

Los tokens de autenticacion y demas secretos deben almacenarse en el almacenamiento seguro del sistema operativo cuando este disponible, **nunca** en almacenamiento plano sin cifrar. Toda comunicacion con el backend por **HTTPS**. El refresh via `POST /auth/refresh` sin exponer tokens en logs de produccion. Al cerrar sesion (`DELETE /auth/logout`) deben eliminarse todos los secretos y datos de sesion locales.

RNF-SEC-002 Control de acceso por rol y dominio

SEC

La navegacion y las llamadas API deben restringirse segun rol: **publico**, **dueno (owner)** o **administrador (admin)**. Un dueno autenticado no debe acceder a pantallas ni endpoints `/admin/*` . El staff admin solo accede al dominio Admin segun permisos de `GET /auth/current_user` . Los intentos no autorizados redirigen a login o informan error **403** de forma clara.

RNF-UX-001 Experiencia de usuario movil

UX

La interfaz debe cumplir patrones moviles de produccion en Android: areas tactiles minimas de **48×48 dp** (Material), inputs con **font-size ≥ 16 px**, estados de carga / vacio / error en todo listado o formulario, navegacion por tabs y stacks segun `mobile-screens.md` , y respeto de **safe areas** y barras del sistema (status bar, navigation bar, recortes de pantalla). Los mensajes deben estar en espanol claro, sin stack traces ni jerga tecnica.

RNF-UX-002 Design system y tema claro/oscuro

UX

La UI debe implementar los tokens centralizados de `mobile-design-tokens.css y src/theme/tokens.ts` , incluyendo **modo claro y modo oscuro** coherentes con `mobile-ui-components.md` . El tema puede seguir la preferencia del sistema o una seleccion manual del usuario.

RNF-REL-001 Tolerancia a fallos

REL

El sistema debe manejar los errores comunes con elegancia — interrupciones de red, timeouts, respuestas **5xx** o token expirado — e informar al usuario, intentando **preservar los datos ingresados** (borrador local, cola de reintentos, no descartar formularios). Los errores **422** deben senalar el campo afectado. Los **401** deben disparar refresh de sesion o re-autenticacion segun corresponda.

RNF-REL-002 Copia de seguridad local y recuperacion

REL

RPO: ante cierre inesperado o fallo del dispositivo, la perdida de datos no sincronizados encolados no debe exceder el **ultimo guardado automatico de borrador** (intervalo maximo recomendado: **5 minutos** mientras se edita, o al perder foco del formulario).

RTO: tras reiniciar la app, el usuario debe poder retomar operaciones pendientes o borradores recuperables en un plazo maximo de **30 segundos** desde el arranque, sin reingresar datos ya capturados localmente.

Los datos clinicos descargados no deben depender de cache permanente sin cifrar; la recuperacion aplica a **cola de sincronizacion**, **borradores** y **estado de sesion** autorizado.

RNF-REL-003 Operacion offline controlada

REL

Sin conexion, la app debe permitir consultar la **ultima copia local** de datos ya sincronizados (cuando exista) y encolar acciones de escritura para envio posterior. Las acciones encoladas deben mostrarse como **pendientes de sincronizacion**. No debe simular exito de operaciones no confirmadas por el servidor.

RNF-PERF-001 Rendimiento en red movil

PERF

Las transiciones entre pantallas principales deben percibirse en **menos de 1 segundo** con conexion normal. Las operaciones largas (upload/download) no deben bloquear la UI; deben mostrar progreso y permitir cancelacion cuando el SO lo permita. Los listados deben soportar paginacion si el API la ofrece.

RNF-ARC-001 Manejo de archivos y multimedia

ARC

La app debe soportar captura y subida de imagenes (fotos de mascota, comprobantes) y descarga o visualizacion de **PDFs medicos** y archivos de laboratorio, conforme a las rutas de upload/download del API. Debe respetar limites de tamano y tipo MIME del backend. Los permisos de camara, galeria y almacenamiento se solicitan en runtime con mensaje contextual.

RNF-PRI-001 Privacidad y datos de salud animal

PRI

Los datos medicos y personales deben tratarse con minima exposicion: sin cache innecesaria en disco, sin logs con informacion identificable en builds de produccion, y con acceso a la politica de privacidad (`GET /pages/privacy`) desde registro y perfil. La eliminacion de cuenta debe purgar datos locales. La auto-sincronizacion (RNF-MOB-002) nunca debe activarse sin consentimiento explicito.

RNF-OBS-001 Observabilidad y registro de errores

OBS

Los errores no controlados deben registrarse con contexto tecnico en entorno de **desarrollo** unicamente. En produccion, el usuario recibe mensajes accionables; el equipo tecnico correlaciona fallos mediante identificadores de error anonimos, sin incluir tokens, contrasenas ni diagnosticos veterinarios completos.

RNF-CMP-001 Compatibilidad de dispositivos Android

CMP

La app debe ser compatible con las versiones de **Android** soportadas por Expo SDK 56 segun `app.json` (API level minimo definido en el proyecto). Debe funcionar en telefonos pequenos (320 px de ancho logico) y grandes, con interaccion tactil completa (sin depender de teclado fisico). Debe validarse manualmente en al menos un dispositivo Android fisico y un emulador durante QA.

RNF-MAINT-001 Mantenibilidad y trazabilidad documental

MAINT

El codigo debe mantener trazabilidad entre requisitos (**RF-***, **RNF-***), rutas API (`mobile-routes.md`), pantallas (`mobile-screens.md`) y componentes (`mobile-ui-components.md`). Los servicios API deben agruparse por dominio en `src/services/` . Cambios en rutas incluidas deben actualizar la documentacion en `docs/` . El demo HTML es referencia visual, no codigo de produccion.

Matriz de trazabilidad resumida

DOMINIO	REQUISITOS FUNCIONALES	RUTAS API (APROX.)
Public	RF-PUB-01 a RF-PUB-05	19
Owner	RF-OWN-01 a RF-OWN-11	54
Admin	RF-ADM-01 a RF-ADM-10	84
No funcionales	RNF-ARQ, MOB, SEC, UX, REL, PERF, ARC, PRI, OBS, CMP, MAINT	—

Apendice — Cobertura endpoint → RF

Mapeo de las **157 rutas** de `mobile-routes.md` a su requisito funcional. Cobertura: **156 / 157** endpoints. Unico endpoint sin RF: `GET /auth/debug` (diagnostico, fuera del alcance funcional).

Public — 19 rutas

METODO · RUTA	RF
pages	
GET /services	**RF-PUB-01**
GET /products	**RF-PUB-02**
GET /checkout	**RF-PUB-02**
GET /contact	**RF-PUB-05**
GET /terms	**RF-PUB-05**
GET /privacy	**RF-PUB-05**
public/products · public/product_orders	
GET /public/products	**RF-PUB-02**
GET /public/products/:id	**RF-PUB-02**
POST /public/product_orders	**RF-PUB-02**
POST /public/product_orders/:id/upload_proof	**RF-PUB-02**
adoption	
GET /adoption	**RF-PUB-03**
GET /adoption/pets	**RF-PUB-03**
GET /adoption/pets/:id	**RF-PUB-03**
sponsorships	
GET /sponsorships	**RF-PUB-04**
GET /sponsorships/:id	**RF-PUB-04**
GET /sponsorships/:id/expenses/:expense_id	**RF-PUB-04**
POST /sponsorships	**RF-PUB-04**
PATCH /sponsorships/:id	**RF-PUB-04**
PUT /sponsorships/:id	**RF-PUB-04**

Owner — 54 rutas

METODO · RUTA	RF
auth	
GET /auth	**RF-OWN-01**
GET /auth/current_user	**RF-OWN-01**
GET /auth/debug	*sin RF (diagnostico)*
POST /auth/login	**RF-OWN-01**
POST /auth/refresh	**RF-OWN-01**
DELETE /auth/logout	**RF-OWN-01**
POST /auth/register_owner	**RF-OWN-02**
POST /auth/forgot_password	**RF-OWN-02**
GET /auth/reset_password/:token	**RF-OWN-02**
POST /auth/reset_password	**RF-OWN-02**
profile	
GET /profile	**RF-OWN-03**
PATCH /profile	**RF-OWN-03**
PATCH /profile/password	**RF-OWN-03**
DELETE /profile	**RF-OWN-03**
dashboard	
GET /dashboard	**RF-OWN-04**
GET /dashboard-summary	**RF-OWN-04**
owner_pages (hubs)	
GET /my-pets	**RF-OWN-05**
GET /my-appointments	**RF-OWN-06**
GET /my-payments	**RF-OWN-07**
GET /my-medical-history	**RF-OWN-08**
GET /my-pets/:pet_id/medical-history	**RF-OWN-08**
GET /my-sponsorships	**RF-OWN-11**
owner_appointments · owner_availability	
GET /owner-appointments-list	**RF-OWN-06**
GET /owner-appointments/:id	**RF-OWN-06**
POST /owner-appointments	**RF-OWN-06**
PATCH /owner-appointments/:id	**RF-OWN-06**
POST /owner-appointments/:id/confirm	**RF-OWN-06**
POST /owner-appointments/:id/cancel	**RF-OWN-06**
GET /owner-available-vets	**RF-OWN-06**
GET /owner-available-days	**RF-OWN-06**
owner_payments	
GET /owner-payments-list	**RF-OWN-07**
POST /owner-payments/:id/register	**RF-OWN-07**
owner_medical_summary · owner_consultations	
GET /owner-medical-summary	**RF-OWN-08**
GET /pets/:pet_id/consultations	**RF-OWN-08**
GET /owner/consultations/:id/export_recipe	**RF-OWN-08**
POST /owner/consultations/:id/complete_treatment	**RF-OWN-08**

METODO · RUTA	RF
pets	
GET /pets	**RF-OWN-05**
GET /pets/new	**RF-OWN-05**
GET /pets/:id	**RF-OWN-05**
GET /pets/:id/edit	**RF-OWN-05**
POST /pets	**RF-OWN-05**
PATCH /pets/:id	**RF-OWN-05**
PUT /pets/:id	**RF-OWN-05**
DELETE /pets/:id	**RF-OWN-05**
POST /pets/:id/photo	**RF-OWN-05**
DELETE /pets/:id/photo	**RF-OWN-05**
owner_medical_profiles · vaccinations · dewormings	
GET /pets/:pet_id/medical_profile	**RF-OWN-09**
GET /pets/:pet_id/vaccinations	**RF-OWN-09**
GET /pets/:pet_id/dewormings	**RF-OWN-09**
owner_lab_exams · owner_medical_reports	
POST /owner/lab_exams/:id/files	**RF-OWN-10**
PATCH /owner/lab_exams/:id/results	**RF-OWN-10**
GET /pets/:pet_id/medical_reports	**RF-OWN-10**
GET /pets/:pet_id/medical_reports/:id	**RF-OWN-10**
GET /pets/:pet_id/medical_reports/:id/download_pdf	**RF-OWN-10**

Admin — 84 rutas

METODO · RUTA	RF
admin/pets	
GET /admin/pets-list	**RF-ADM-01**
GET /admin/pets/:id	**RF-ADM-01**
POST /admin/pets	**RF-ADM-01**
PATCH /admin/pets/:id	**RF-ADM-01**
PUT /admin/pets/:id	**RF-ADM-01**
DELETE /admin/pets/:id	**RF-ADM-01**
POST /admin/pets/:id/photo	**RF-ADM-01**
DELETE /admin/pets/:id/photo	**RF-ADM-01**
admin/adoptions	
GET /admin/adoptions-list	**RF-ADM-02**
GET /admin/adoptions/:id	**RF-ADM-02**
POST /admin/adoptions	**RF-ADM-02**
admin/consultations	
GET /admin/consultations-list	**RF-ADM-03**
GET /admin/consultations/:id	**RF-ADM-03**
GET /admin/consultations/:id/export_recipe	**RF-ADM-03**
POST /admin/consultations	**RF-ADM-03**
POST /admin/consultations/:id/complete_treatment	**RF-ADM-03**
PATCH /admin/consultations/:id	**RF-ADM-03**
PUT /admin/consultations/:id	**RF-ADM-03**
DELETE /admin/consultations/:id	**RF-ADM-03**
admin/prescriptions (consultations anidadas)	
GET .../prescriptions	**RF-ADM-04**
GET .../prescriptions/:id	**RF-ADM-04**
POST .../prescriptions	**RF-ADM-04**
PATCH .../prescriptions/:id	**RF-ADM-04**
PUT .../prescriptions/:id	**RF-ADM-04**
DELETE .../prescriptions/:id	**RF-ADM-04**
admin/lab_exams (consultations anidadas)	
GET .../lab_exams	**RF-ADM-04**
GET .../lab_exams/:id	**RF-ADM-04**
POST .../lab_exams	**RF-ADM-04**
POST .../lab_exams/:id/files	**RF-ADM-04**
PATCH .../lab_exams/:id	**RF-ADM-04**
PUT .../lab_exams/:id	**RF-ADM-04**
DELETE .../lab_exams/:id	**RF-ADM-04**
DELETE .../lab_exams/:id/files/:file_id	**RF-ADM-04**
admin/vaccinations	
GET /admin/vaccinations-list	**RF-ADM-05**
GET /admin/vaccinations/:id	**RF-ADM-05**
POST /admin/vaccinations	**RF-ADM-05**
PATCH /admin/vaccinations/:id	**RF-ADM-05**

METODO · RUTA	RF
PUT /admin/vaccinations/:id	**RF-ADM-05**
DELETE /admin/vaccinations/:id	**RF-ADM-05**
admin/dewormings	
GET /admin/dewormings-list	**RF-ADM-05**
GET /admin/dewormings/:id	**RF-ADM-05**
POST /admin/dewormings	**RF-ADM-05**
PATCH /admin/dewormings/:id	**RF-ADM-05**
PUT /admin/dewormings/:id	**RF-ADM-05**
DELETE /admin/dewormings/:id	**RF-ADM-05**
admin/vaccination_schedules	
GET /admin/vaccination-schedules-list	**RF-ADM-05**
GET /admin/vaccination_schedules/:id	**RF-ADM-05**
POST /admin/vaccination_schedules	**RF-ADM-05**
POST /admin/vaccination_schedules/:id/complete_item	**RF-ADM-05**
PATCH /admin/vaccination_schedules/:id	**RF-ADM-05**
PUT /admin/vaccination_schedules/:id	**RF-ADM-05**
DELETE /admin/vaccination_schedules/:id	**RF-ADM-05**
admin/appointments	
GET /admin/appointments-list	**RF-ADM-06**
GET /admin/appointments/:id	**RF-ADM-06**
GET /admin/appointments/available-days	**RF-ADM-06**
GET /admin/appointments/available-vets	**RF-ADM-06**
GET /admin/appointments/time-slots	**RF-ADM-06**
POST /admin/appointments	**RF-ADM-06**
PATCH /admin/appointments/:id	**RF-ADM-06**
PUT /admin/appointments/:id	**RF-ADM-06**
DELETE /admin/appointments/:id	**RF-ADM-06**
admin/payments	
GET /admin/payments-list	**RF-ADM-07**
GET /admin/payments/:id	**RF-ADM-07**
POST /admin/payments/:id/register	**RF-ADM-07**
PATCH /admin/payments/:id	**RF-ADM-07**
PUT /admin/payments/:id	**RF-ADM-07**
admin/medical_profiles	
GET /admin/pets/:pet_id/medical_profile	**RF-ADM-08**
PATCH /admin/pets/:pet_id/medical_profile	**RF-ADM-08**
PUT /admin/pets/:pet_id/medical_profile	**RF-ADM-08**
admin/owner_pets	
GET /admin/owners/:owner_id/pets/:id	**RF-ADM-09**
POST /admin/owners/:owner_id/pets	**RF-ADM-09**
PATCH /admin/owners/:owner_id/pets/:id	**RF-ADM-09**
PUT /admin/owners/:owner_id/pets/:id	**RF-ADM-09**
POST /admin/owners/:owner_id/pets/:id/photo	**RF-ADM-09**

METODO · RUTA	RF
DELETE /admin/owners/:owner_id/pets/:id/photo	**RF-ADM-09**
admin/medical_reports	
GET /admin/medical_reports	**RF-ADM-10**
GET /admin/medical_reports/:id	**RF-ADM-10**
GET /admin/medical_reports/:id/export_csv	**RF-ADM-10**
GET /admin/medical_reports/:id/export_json	**RF-ADM-10**
GET /admin/medical_reports/:id/export_pdf	**RF-ADM-10**
GET /admin/medical_reports/generate_pdf	**RF-ADM-10**
POST /admin/medical_reports	**RF-ADM-10**
POST /admin/medical_reports/:id/email	**RF-ADM-10**
POST /admin/medical_reports/import	**RF-ADM-10**

PawCare Mobile — Demo UI

Solo diseno · Sin funcionalidad · Basado en `mobile-routes.md` · Tokens en `mobile-design-tokens.css`

Public — Descubrimiento y compra

Servicios, tienda, adopcion e informacion legal. Sin autenticacion.

Servicios

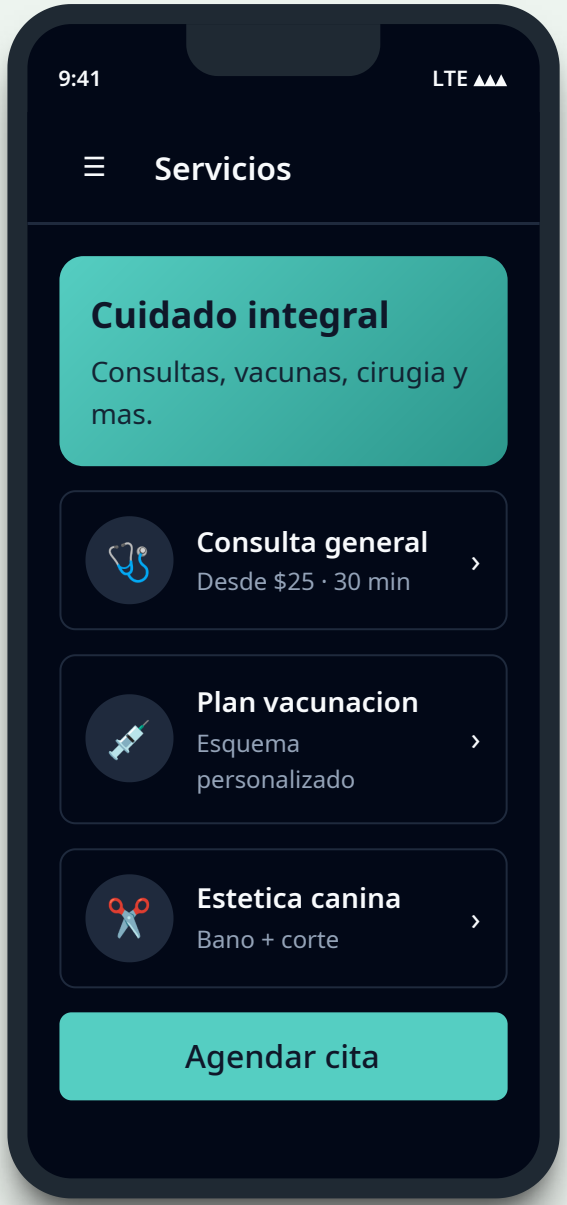
`GET /services`

Public

CLARO



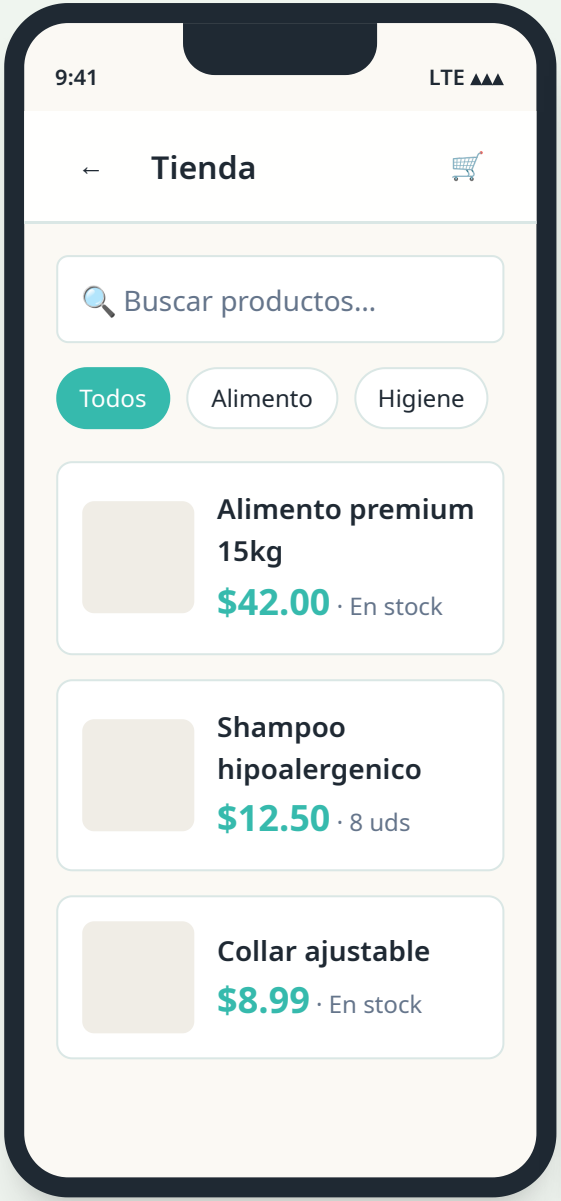
OSCURO



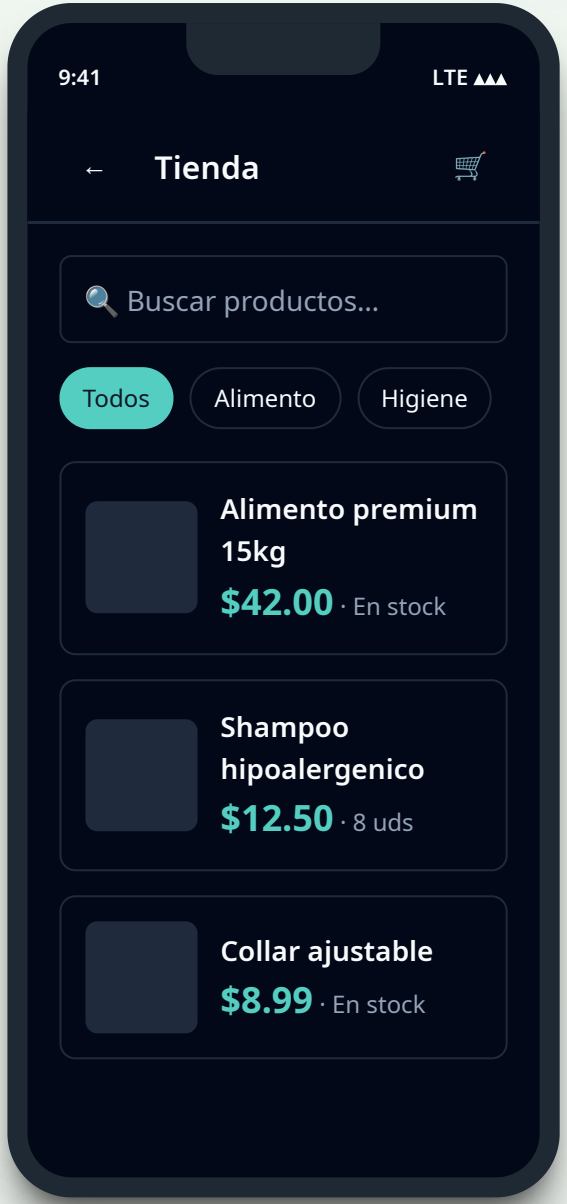
Productos

`GET /public/products`

CLARO



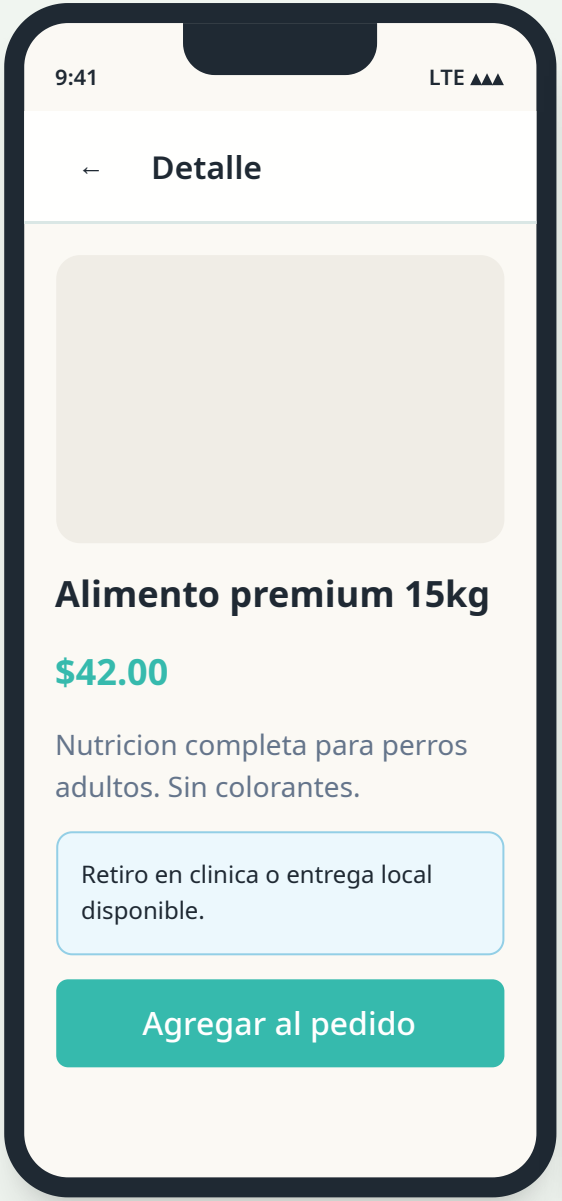
OSCURO



Detalle producto

`GET /public/products/:id`

CLARO



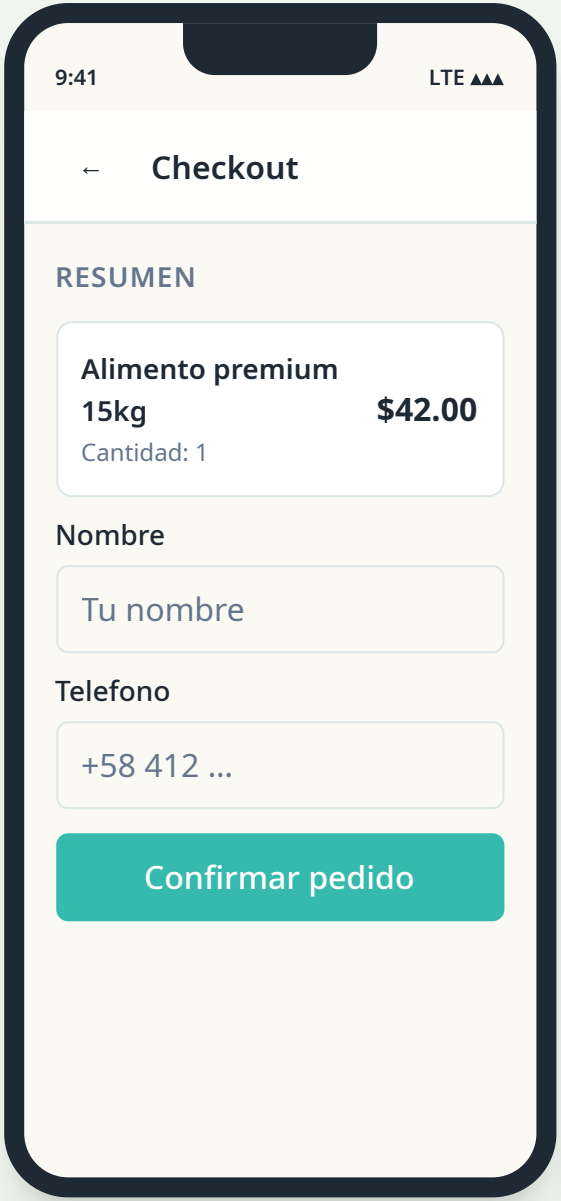
OSCURO



Checkout

`POST /public/product_orders`

CLARO



OSCURO



Adopcion

`GET /adoption · /adoption/pets`

CLARO



OSCURO



Detalle adopcion

`GET /adoption/pets/:id`

CLARO



OSCURO



Contacto

`GET /pages/contact`

CLARO



OSCURO



Owner — Autenticacion

Flujo de acceso para duenos de mascota.

Bienvenida

`GET /auth`

CLARO



OSCURO



Login

`POST /auth/login`

CLARO



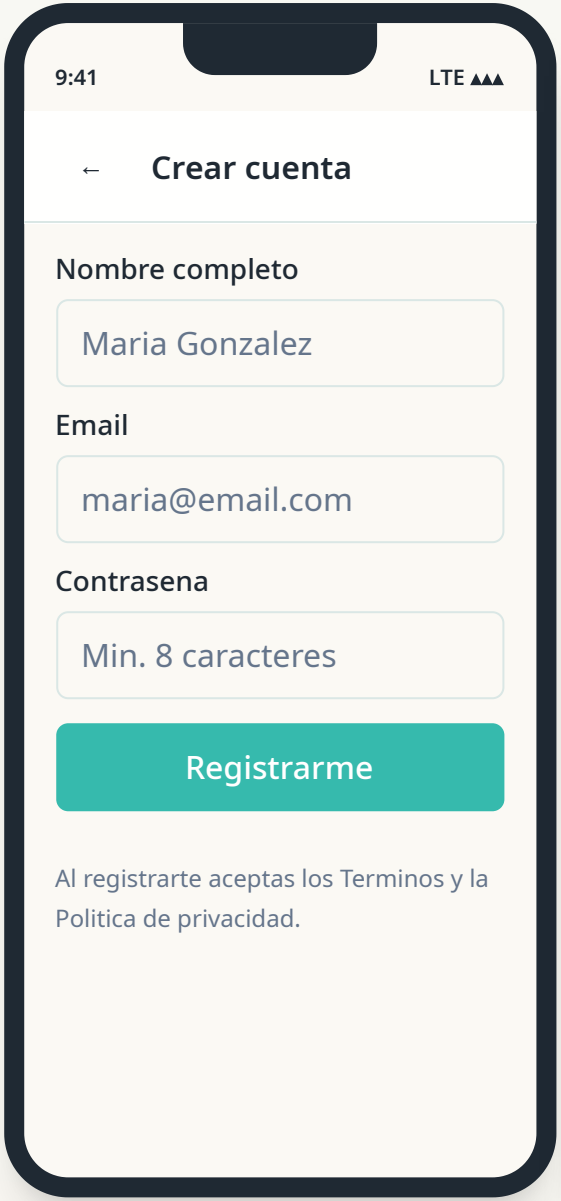
OSCURO



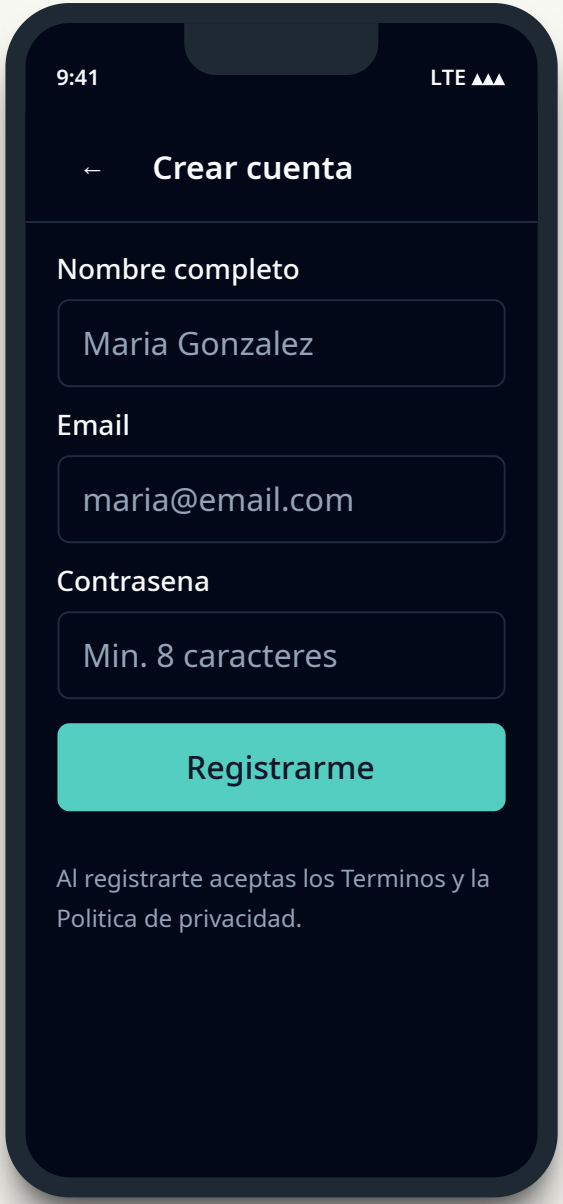
Registro

`POST /auth/register_owner`

CLARO



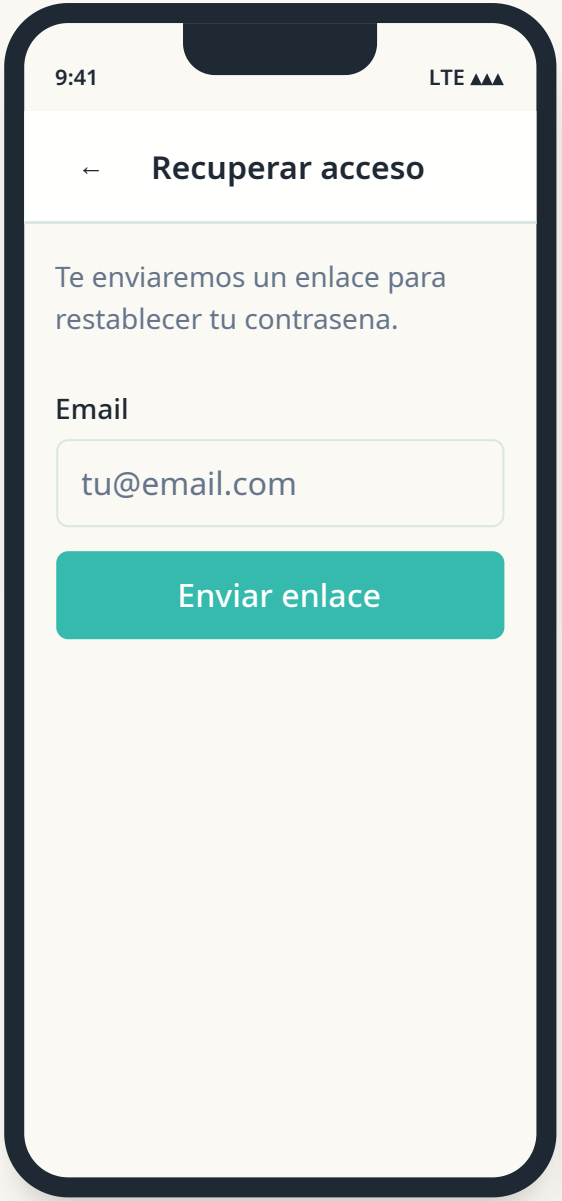
OSCURO



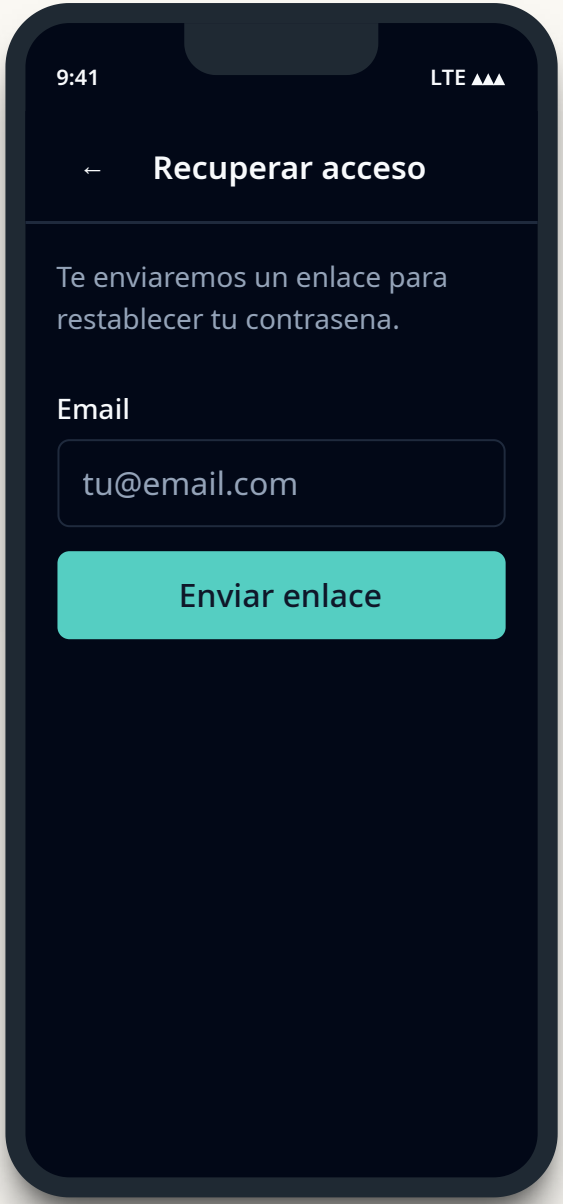
Recuperar contrasena

`POST /auth/forgot_password`

CLARO



OSCURO



Owner — App principal

Dashboard, mascotas, citas, pagos y perfil.

Dashboard

`GET /dashboard · /dashboard-summary`

CLARO



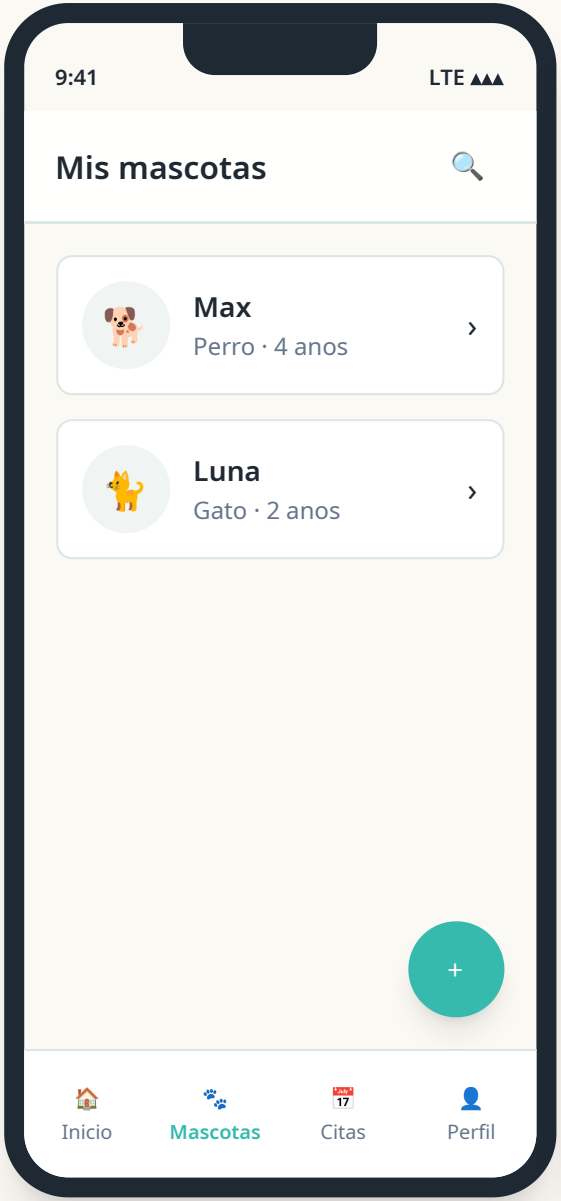
OSCURO



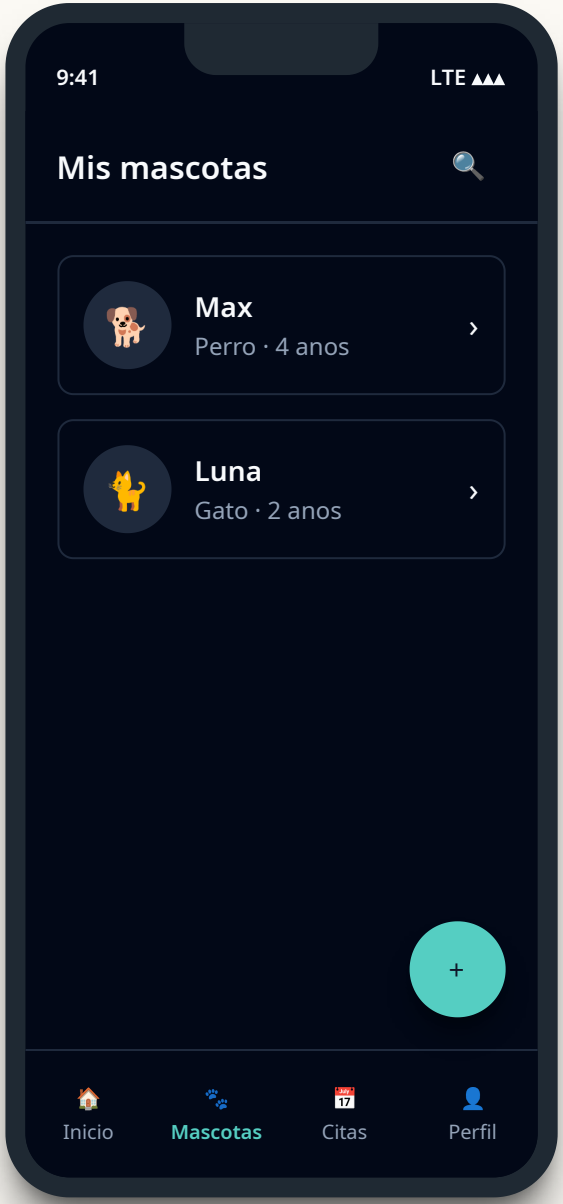
Mis mascotas

`GET /my-pets · /pets`

CLARO



OSCURO



Detalle mascota

`GET /pets/:id`

CLARO



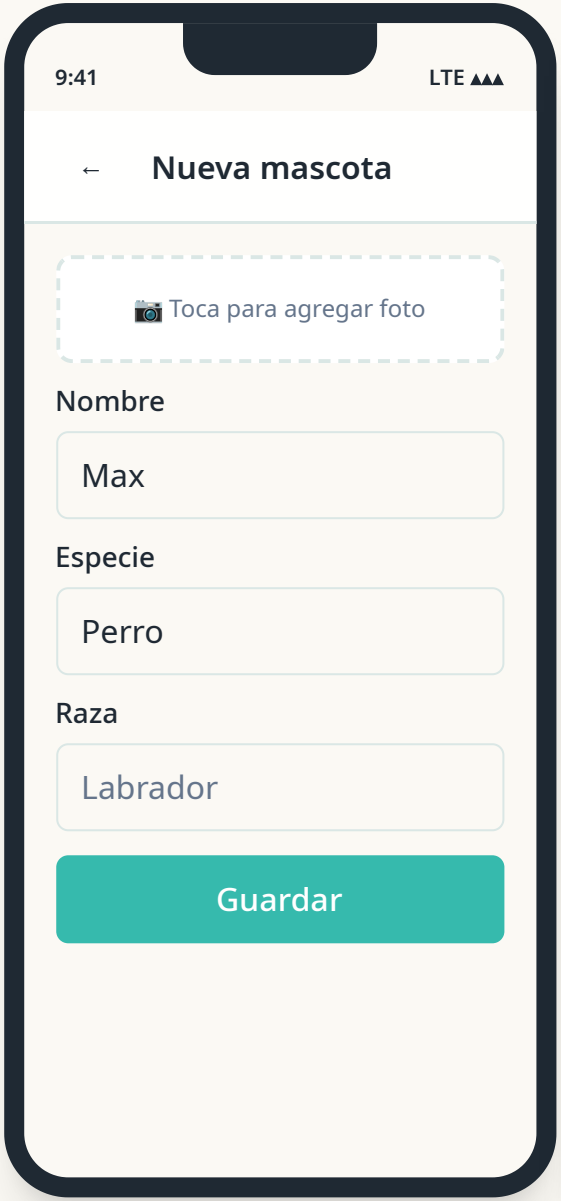
OSCURO



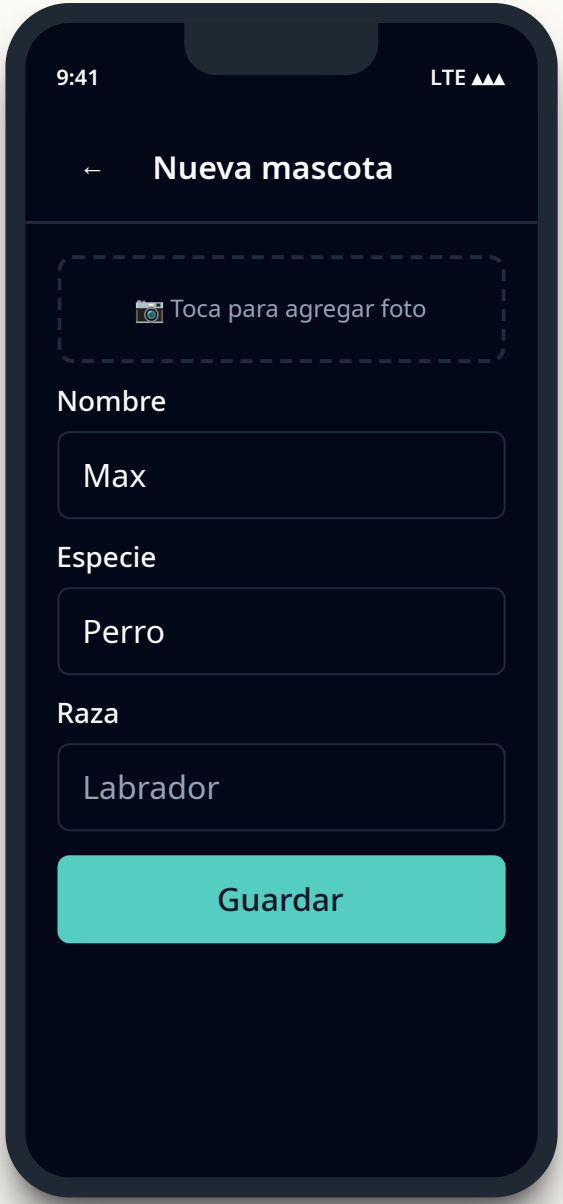
Nueva mascota

`POST /pets · POST /pets/:id/photo`

CLARO



OSCURO



Mis citas

`GET /owner-appointments-list`

CLARO



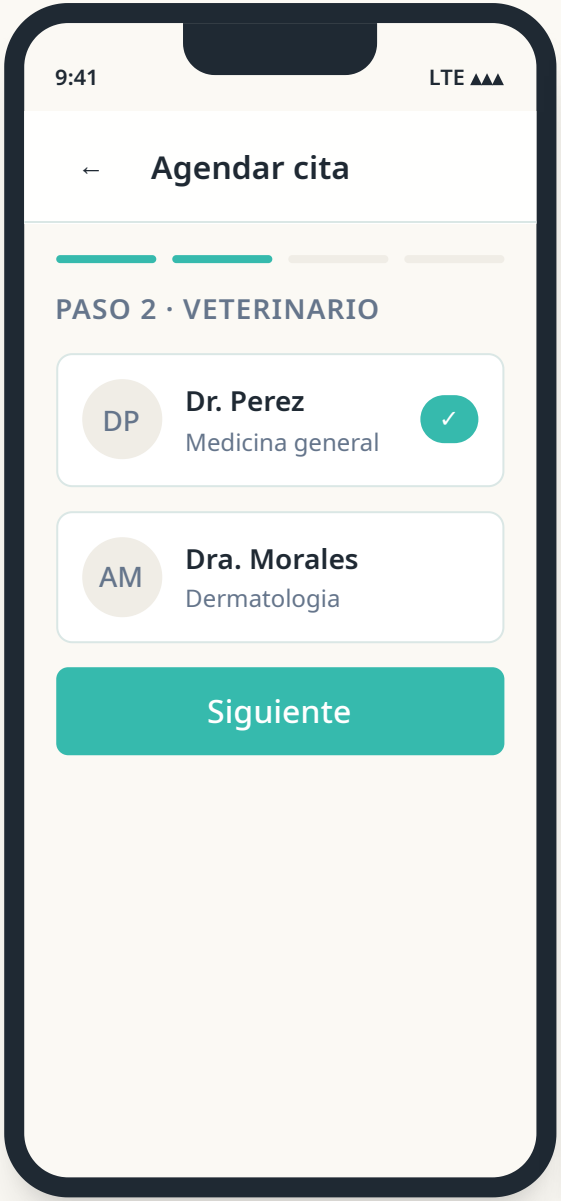
OSCURO



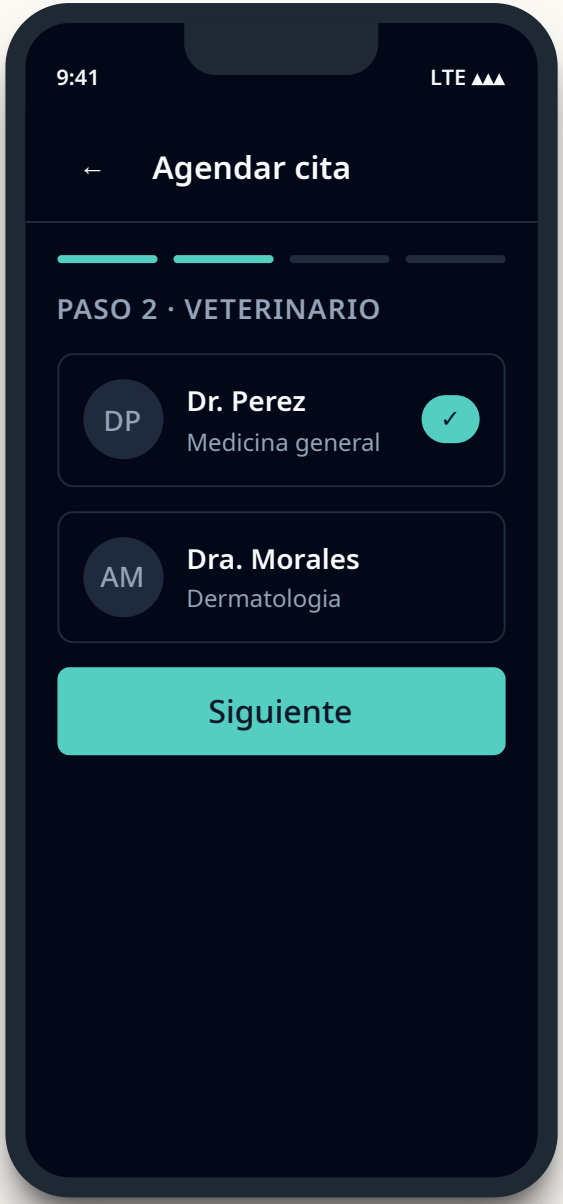
Agendar cita

`POST /owner-appointments · availability`

CLARO



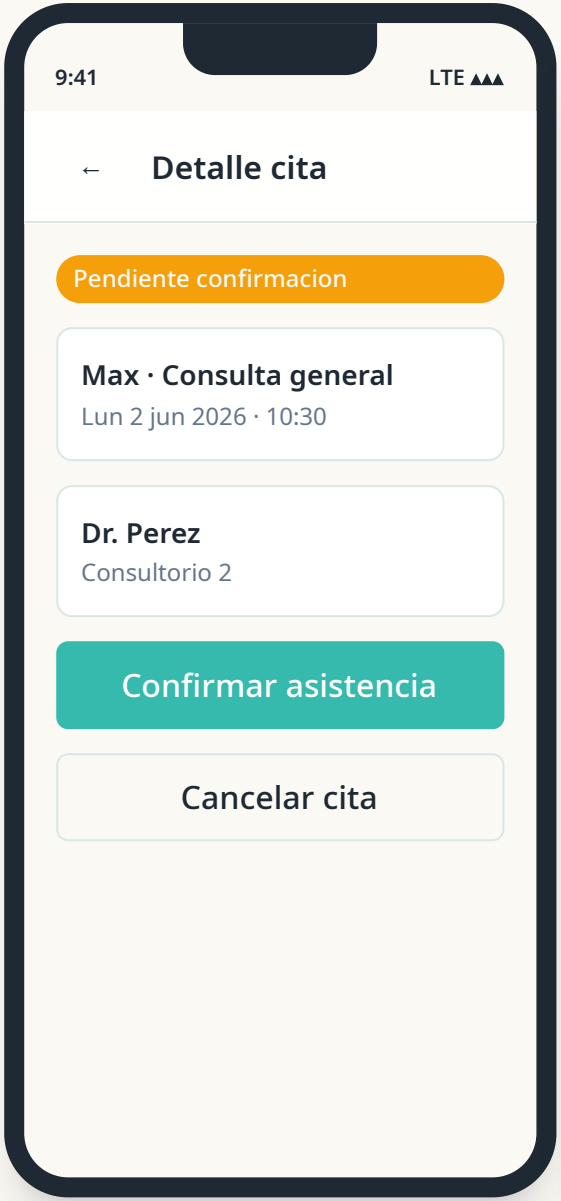
OSCURO



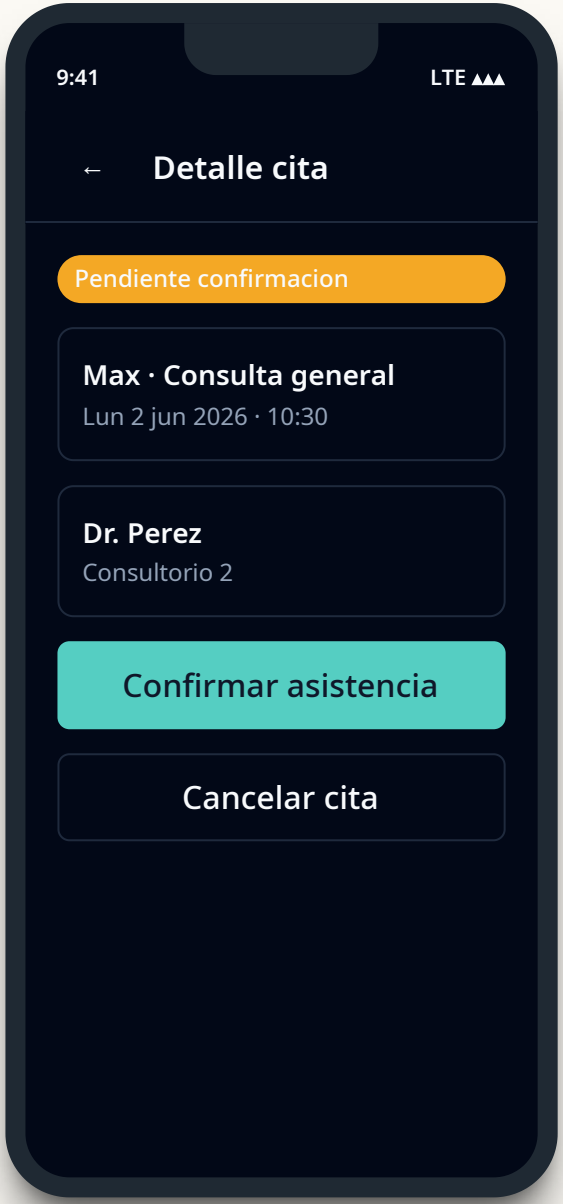
Detalle cita

`GET /owner-appointments/:id · confirm/cancel`

CLARO



OSCURO



Pagos

`GET /owner-payments-list · register`

CLARO



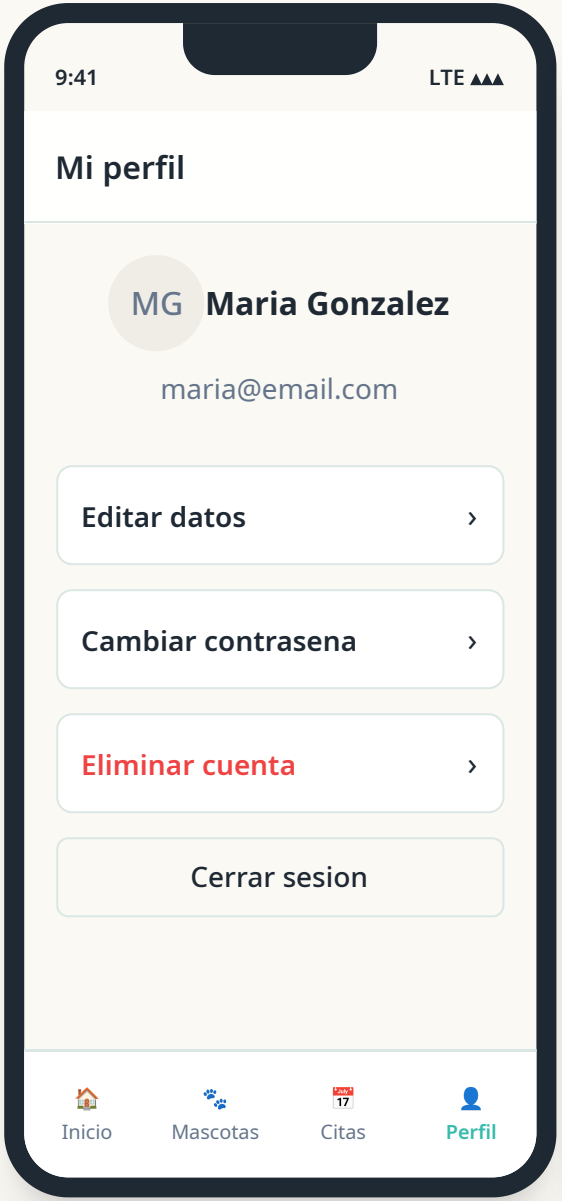
OSCURO



Mi perfil

`GET /profile · PATCH /profile`

CLARO



OSCURO



Owner — Historial medico

Consultas, vacunas, laboratorio y reportes por mascota.

Historial medico

`GET /my-medical-history · /owner-medical-summary`

CLARO



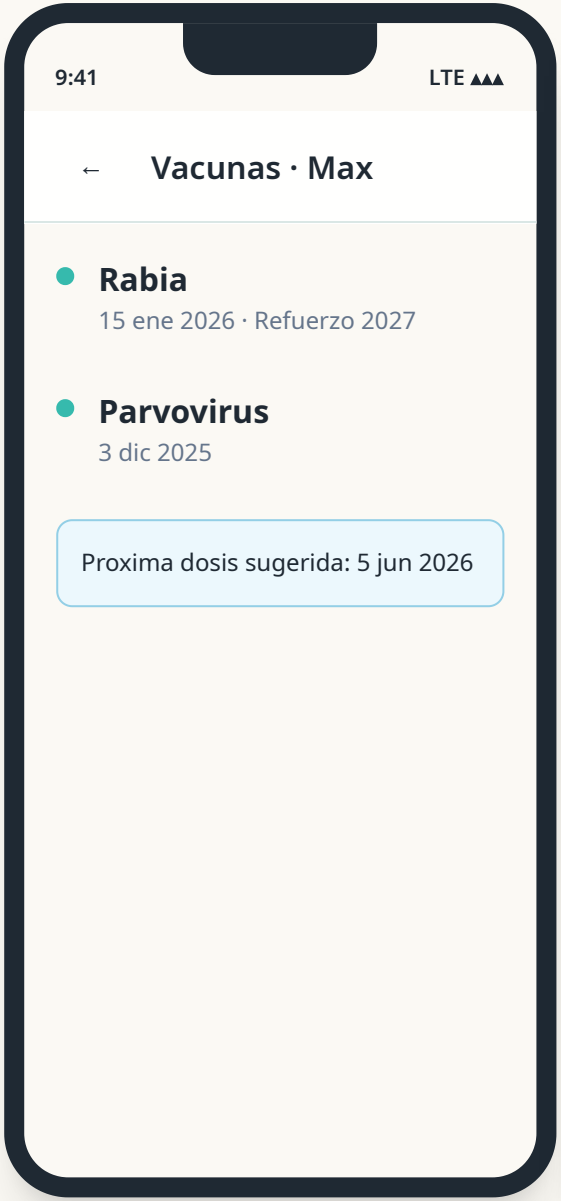
OSCURO



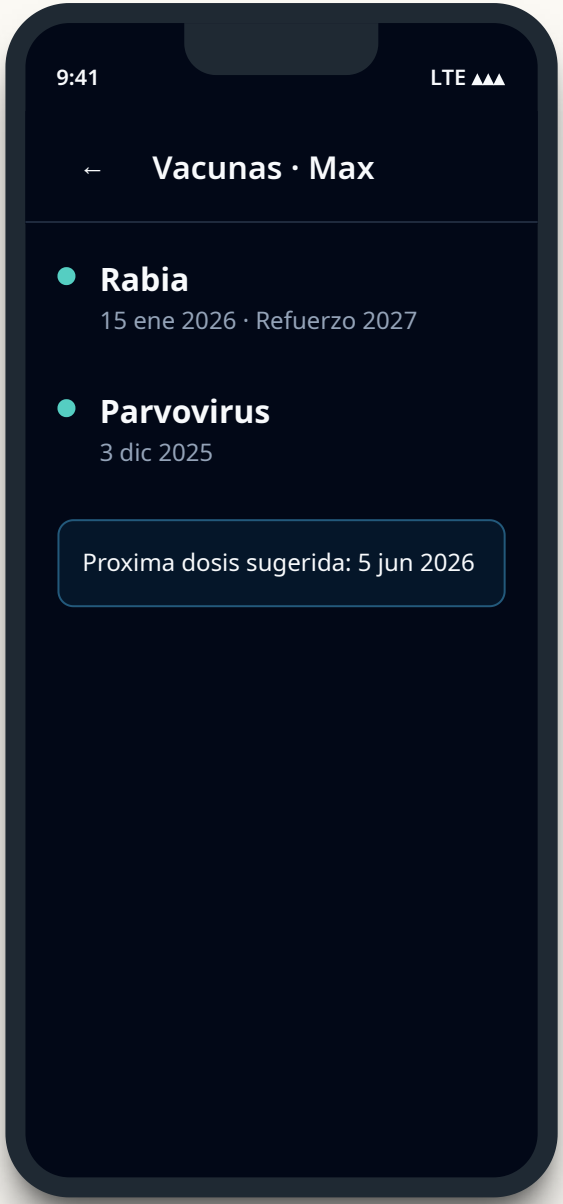
Vacunas

`GET /pets/:pet_id/vaccinations`

CLARO



OSCURO



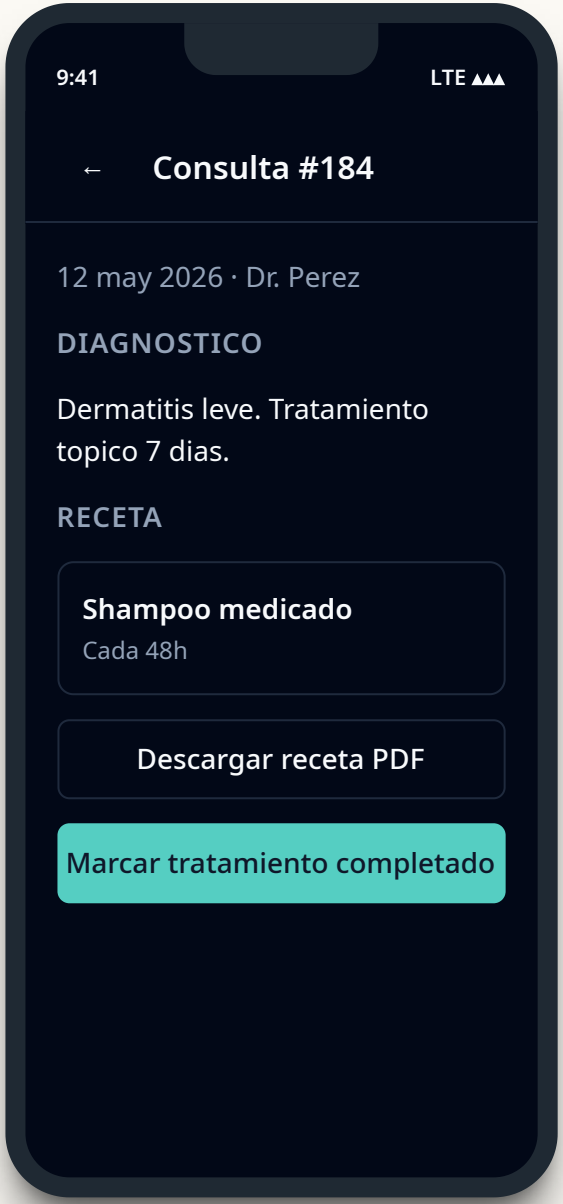
Detalle consulta

`GET /pets/:pet_id/consultations · export_recipe`

CLARO

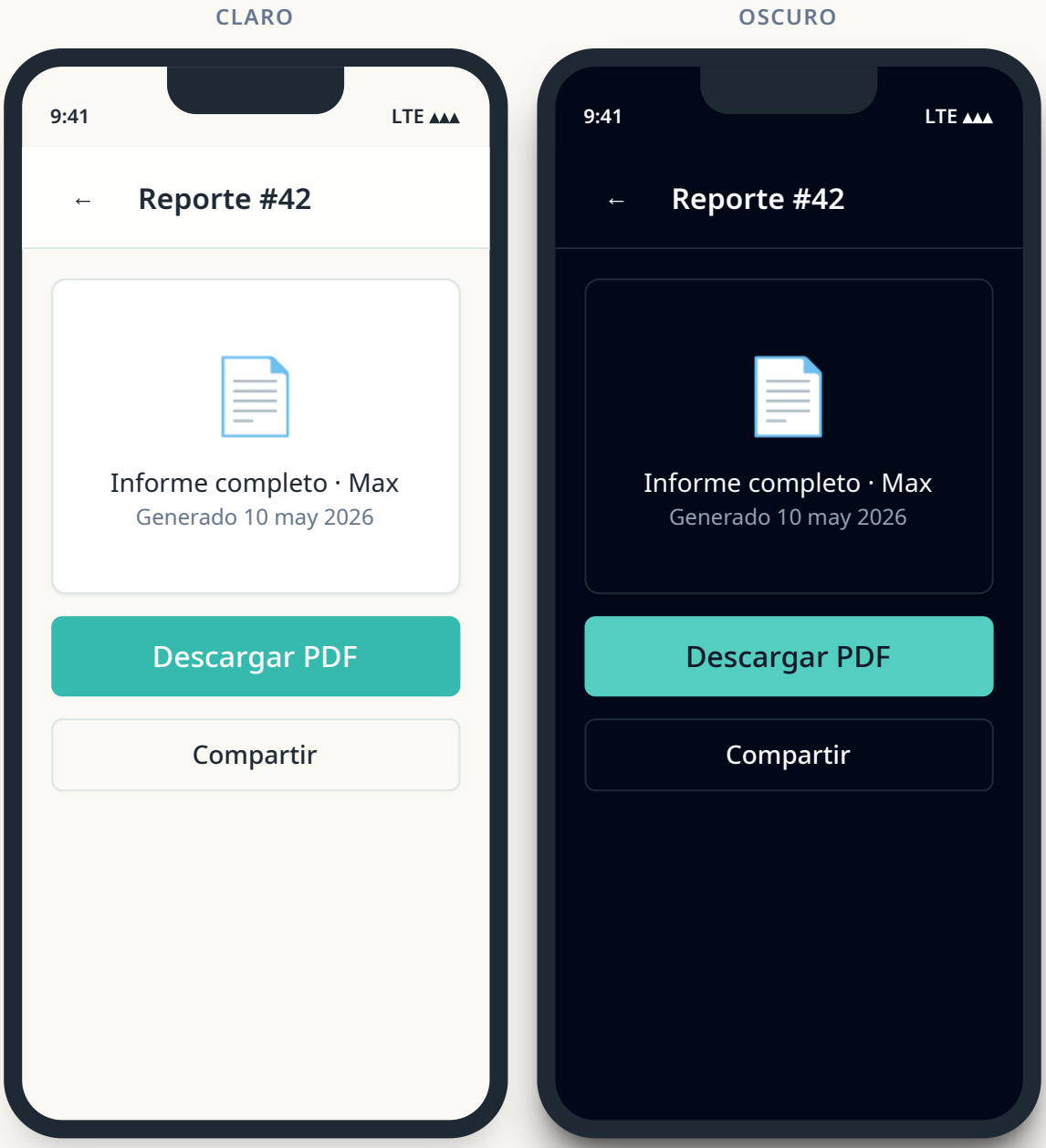


OSCURO



Reporte medico

`GET /pets/:pet_id/medical_reports/:id`



Admin — Panel administrativo

Flujos de staff: pacientes, consultas, agenda, pagos y reportes.

Admin inicio

–

Admin

CLARO



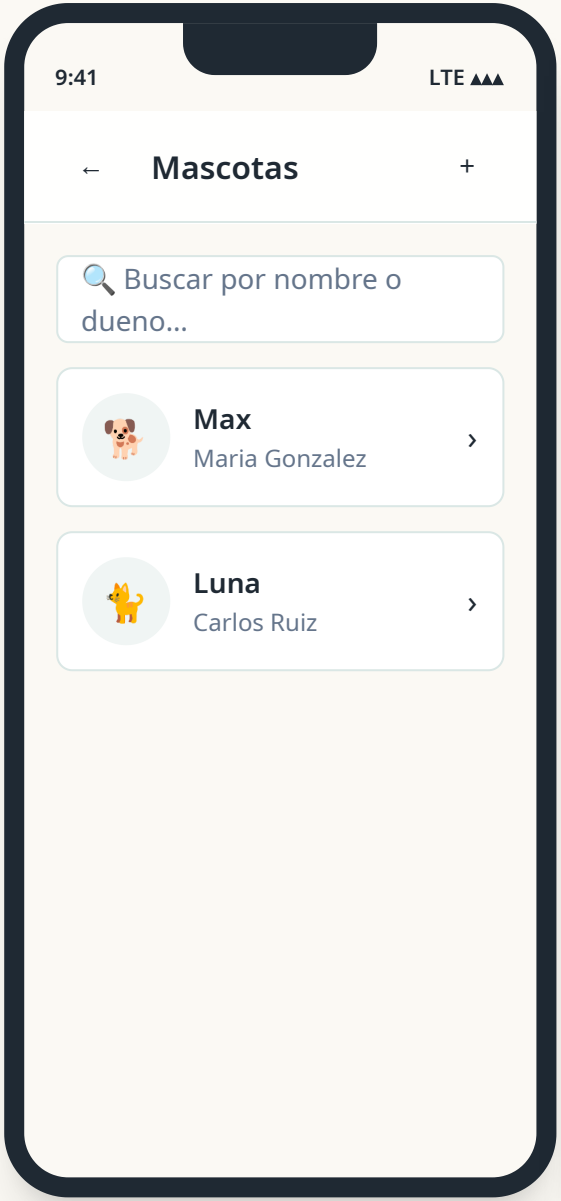
OSCURO



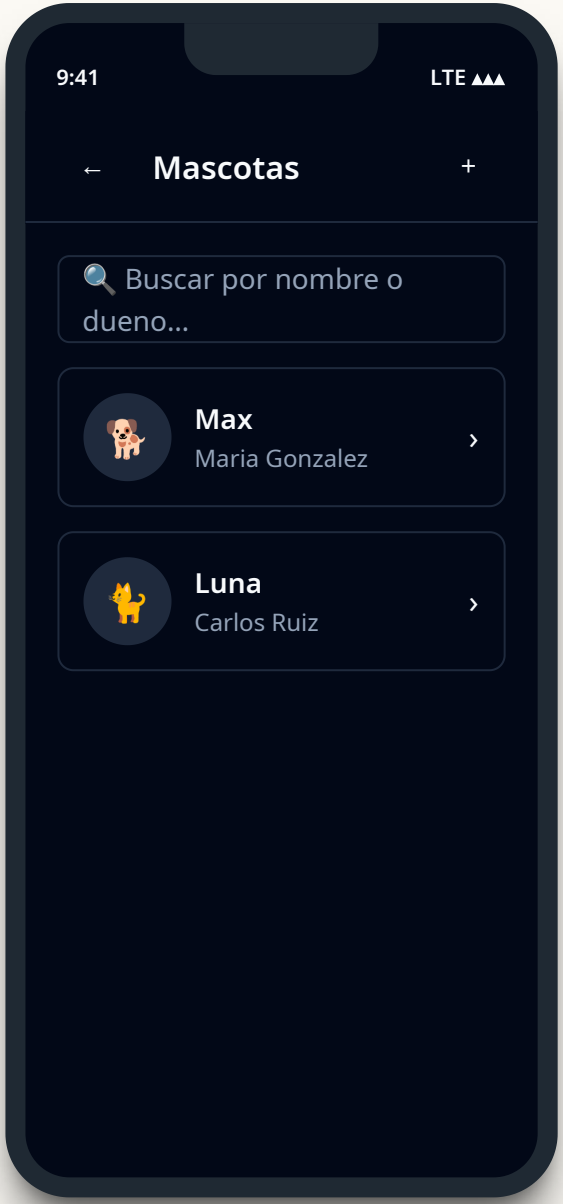
Mascotas admin

`GET /admin/pets-list`

CLARO



OSCURO



Consultas admin

`GET /admin/consultations-list`

CLARO



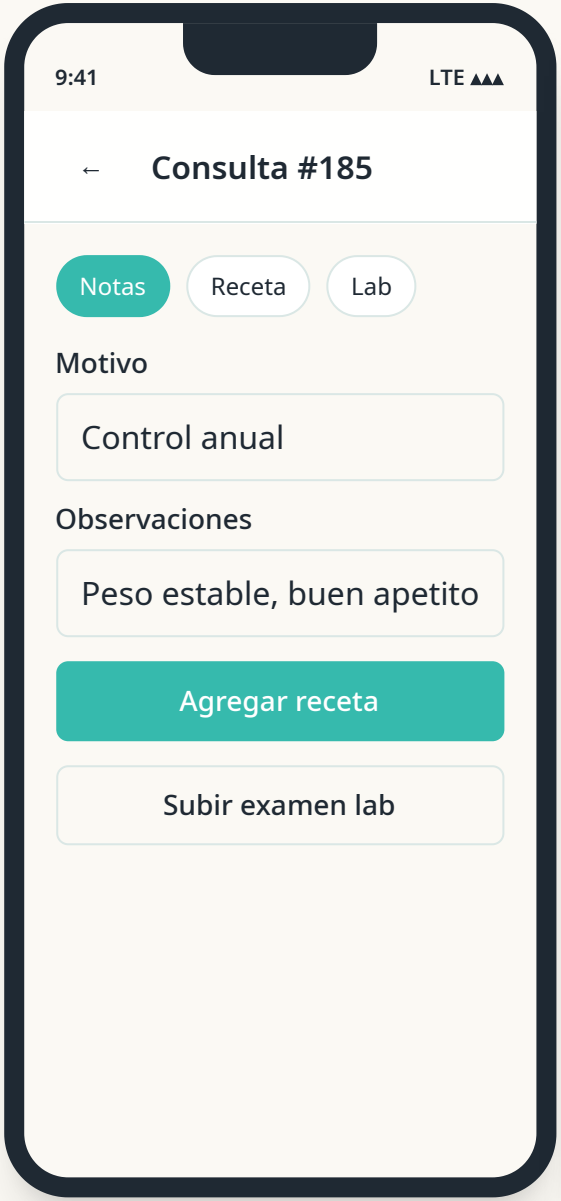
OSCURO



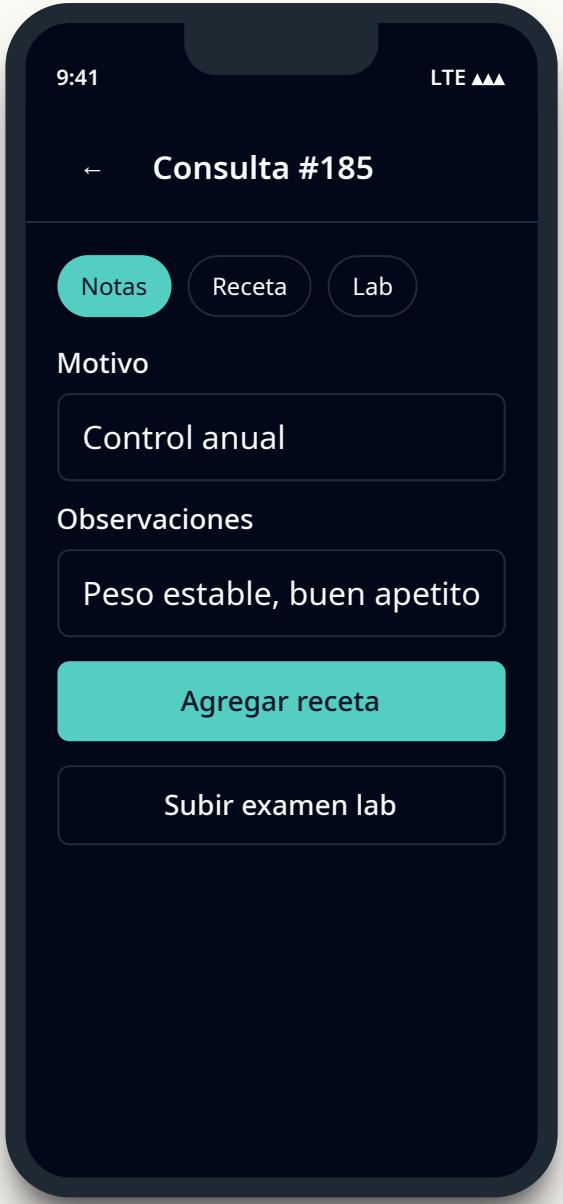
Detalle consulta admin

`prescriptions · lab_exams`

CLARO



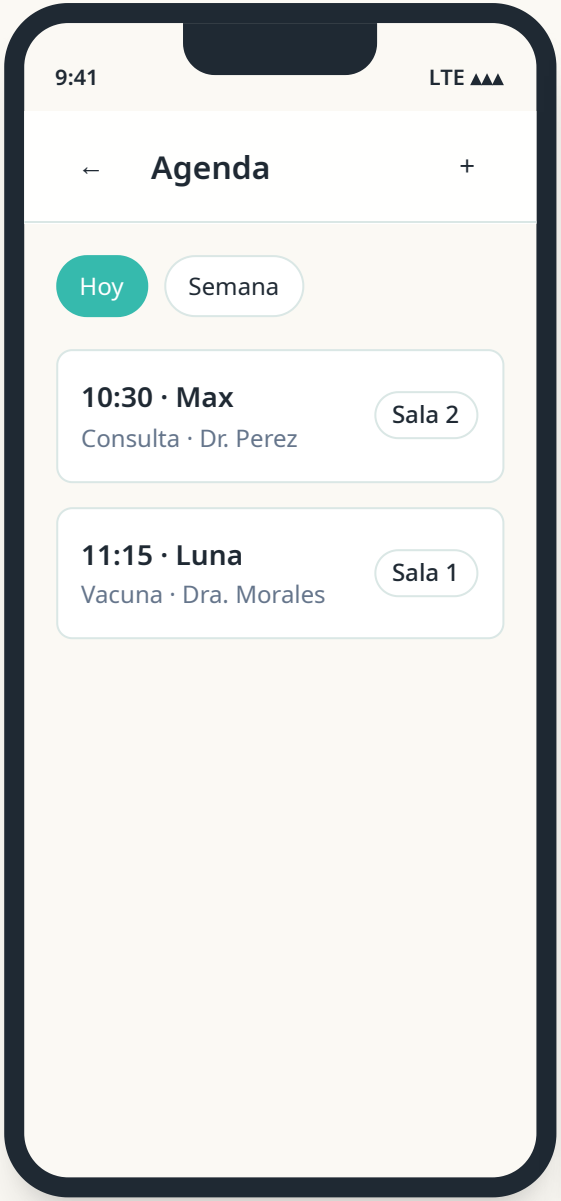
OSCURO



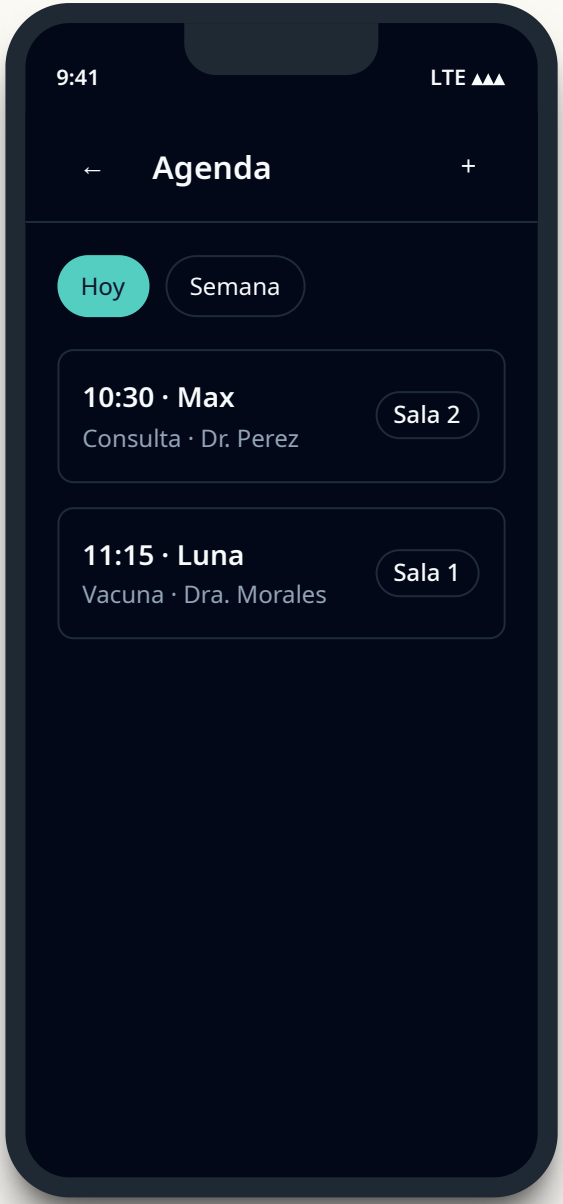
Agenda admin

`GET /admin/appointments-list`

CLARO



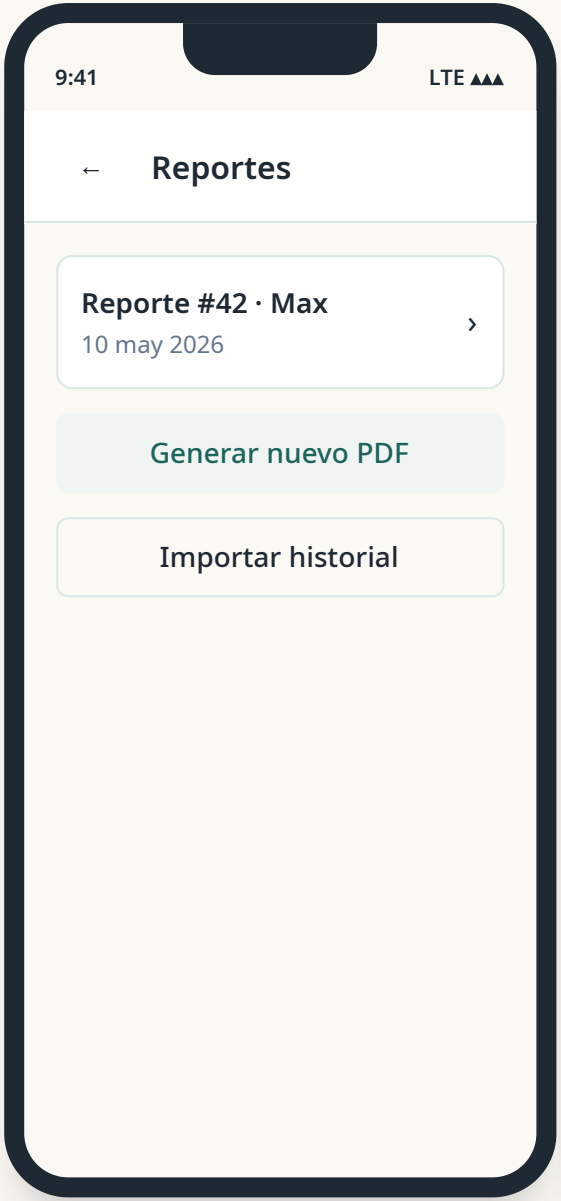
OSCURO



Reportes medicos admin

`GET /admin/medical_reports`

CLARO



OSCURO

